

DATA-260 Agentic AI - Homework 5

Pragnaya Priyadarshini

GitHub repository: <https://github.com/pragnayap/data260-3170>

Section 0 - Personal Configuration and Domain

Value	Derivation	This submission
SID4	Last four digits of SJSU Student ID	3170
PORT_BASE	$8000 + (\text{SID4} \bmod 900)$	8470
PREFIX	"s" + SID4	s3170
SEED	SID4	3170
VERIFY_SEED	$260000 + \text{SID4}$	263170
DOMAIN_ID	$\text{SID4} \bmod 8$	2 - Municipal Transit Incidents

Run environment	Value
Hardware	MacBook Pro, Intel Core i9-9880H @ 2.30 GHz (8 cores), 16 GB RAM, macOS 15.7.9
Local model	qwen3:8b via Ollama (temperature 0.0, reasoning disabled)
Python / Node	Python 3.12.13 (.venv), Node 24.21.0
Database	MySQL 9.4.0 (Homebrew), schema s3170_rel
MCP SDK	mcp 1.30.0 (FastMCP over STDIO)
Tagged commit hash	b41f64beb83cea904dc2a31049c39db2146c41a5

Part 1 - Backend Extension and Redux Client

HW5 extends the HW4 FastAPI + MySQL service rather than starting a new one. The related entity is the existing routes table, which HW4 used only as N+1 test data; HW5 gives it the full endpoint set and adds the relationship query.

I. Database

Both tables were extended to the required shape. incidents is the primary domain entity; routes is the related entity (the "author" role in the book/author example).

Requirement	routes (related)	incidents (primary)
Primary key	id	id
Primary text field	route_name	route_id
Secondary text field	agency	location
Unique field	route_code - format R-NNNN	incident_code - format INC-NNNN-NNNNN
Numeric field with default	-	riders_affected, default 0, NOT NULL
Foreign key	-	related_route_id → routes.id, ON DELETE RESTRICT
Timestamps	created_at, updated_at	created_at, updated_at

Table 1.1 - the schema after the HW5 migration.

Deletion policy. A route that still has incidents cannot be deleted. This is enforced twice: ON DELETE RESTRICT at the MySQL level, so the invariant holds even for direct SQL, and an explicit pre-check in the router so the API answers 409 Conflict with a readable message instead of surfacing a database error as a 500. No cascade is implemented: cascade-deleting a route would silently destroy every incident report filed against it, which is the wrong default for an audit-style domain.

Passwords. No plain-text password is stored. The users table holds salt\$hash produced by PBKDF2-HMAC-SHA256 with 200,000 iterations and a per-user salt, carried over from HW4.

Code 1.1 - models.py: the Incident table (unique code, numeric default, RESTRICT foreign key, timestamps)

```
class Incident(Base):
    """The primary domain entity.

    Naming note for the grader: `route_id` is the assignment's free-text
    primary field (the rider-visible line name, e.g. "Line 22"), carried over
    from DOMAIN_SCHEMA.md. The actual foreign key to the routes table is
    `related_route_id`. They are deliberately different columns -- one is
    display text the submitter typed, the other is a referential integrity
    constraint.
    """

    __tablename__ = "incidents"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)

    # Unique field. Format validated at the API boundary with ^INC-\\d{4}-\\d{5}$.
    incident_code: Mapped[str] = mapped_column(String(24), unique=True, index=True)

    route_id: Mapped[str] = mapped_column(String(80), index=True) # primary field
    location: Mapped[str] = mapped_column(String(160), index=True) # secondary field
    submitter_email: Mapped[str] = mapped_column(String(255))
    description: Mapped[str] = mapped_column(Text)
    category: Mapped[str] = mapped_column(String(40), index=True)

    # Numeric field with a sensible default: an incident reported with no
    # rider-count estimate is recorded as affecting 0, not NULL, so aggregate
    # queries (HW5 Part 2B's aggregate tool) never have to special-case NULL.
    riders_affected: Mapped[int] = mapped_column(
        Integer, default=0, server_default="0", nullable=False
    )

    related_route_id: Mapped[int] = mapped_column(
        ForeignKey("routes.id", ondelete="RESTRICT"), index=True
    )

    created_at: Mapped[dt.datetime] = mapped_column(DateTime, default=_utcnow)
    updated_at: Mapped[dt.datetime] = mapped_column(
        DateTime, default=_utcnow, onupdate=_utcnow
    )

    route: Mapped["Route"] = relationship(back_populates="incidents")
```

[SCREENSHOT 1.1] MySQL: DESCRIBE incidents; and DESCRIBE routes;

```
Pragnaya Priyadarshini data260-3170 % mysql -u root -p s3170_rel -e "DESCRIBE incidents; DESCRIBE routes;"
Enter password:
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | int  | NO   | PRI | NULL    | auto_increment |
| incident_code | varchar(24) | NO | UNI | NULL    |
| route_id | varchar(80) | NO | MUL | NULL    |
| location | varchar(160) | NO | MUL | NULL    |
| submitter_email | varchar(255) | NO | | NULL    |
| description | text | NO | | NULL    |
| category | varchar(40) | NO | MUL | NULL    |
| riders_affected | int | NO | | 0 |
| related_route_id | int | NO | MUL | NULL    |
| created_at | datetime | NO | | NULL    |
| updated_at | datetime | NO | | NULL    |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | int  | NO   | PRI | NULL    | auto_increment |
| route_code | varchar(20) | NO | UNI | NULL    |
| route_name | varchar(120) | NO | | NULL    |
| agency | varchar(80) | NO | | NULL    |
| route_type | varchar(40) | NO | | NULL    |
| created_at | datetime | NO | | NULL    |
| updated_at | datetime | NO | | NULL    |
+-----+-----+-----+-----+-----+-----+
Pragnaya Priyadarshini data260-3170 %
```

Figure 1.1 - MySQL: DESCRIBE incidents; and DESCRIBE routes;

[SCREENSHOT 1.2] Output of python -u code/seed_hw05.py:

```
ModuleNotFoundError: No module named 'pymysql'
Pragnaya Priyadarshini data260-3170 % deactivate
source .venv/bin/activate
hash -r
which python

/Users/ritchit/Desktop/DATA 260--Agentic AI-Homework/data260-3170/.venv/bin/python
Pragnaya Priyadarshini data260-3170 % python -u code/seed_hw05.py

[14:25:47] seed_hw05.py starting, SEED=3170
[14:25:47] dropping + recreating incidents/routes (users/sessions untouched)...
[14:25:48] inserting 200 routes...
[14:25:48] generating 5000 incidents...
[14:25:48] inserting 5000 incidents...
[14:25:48] done in 0.7s -- routes=200 incidents=5000
unique fields: routes.route_code (R-NNNN), incidents.incident_code (INC-3170-NNNNN)
FK incidents.related_route_id -> routes.id ON DELETE RESTRICT
Pragnaya Priyadarshini data260-3170 %
```

Figure 1.2 - Output of python -u code/seed_hw05.py: 200 routes, 5000 incidents and the two schema lines

II. API Implementation

Both entities have the full endpoint set plus one relationship endpoint. Every route requires a live session (s3170_db_session cookie from POST /api/auth/login); an unauthenticated request returns 401.

Method and path	Purpose	Codes
GET /api/routes	List related entity, paginated	200
GET /api/routes/{id}	Retrieve one, with incident_count	200, 404
POST /api/routes	Create	201, 409, 422
PUT /api/routes/{id}	Update	200, 404, 409, 422
DELETE /api/routes/{id}	Delete - refused while incidents remain	204, 404, 409
GET /api/routes/{id}/incidents	Relationship query	200, 404
GET /api/incidents	List primary entity, paginated	200
GET /api/incidents/{id}	Retrieve one	200, 404
POST /api/incidents	Create	201, 404, 409, 422
PUT /api/incidents/{id}	Update	200, 404, 409, 422
DELETE /api/incidents/{id}	Delete	204, 404

Table 1.2 - eleven operations. The relationship endpoint returns every incident associated with one route.

Error semantics. A malformed body is 422: Pydantic rejects the unique field's format before any query runs. A reference to a record that does not exist is 404, including a create that names a non-existent related_route_id - without that check MySQL would raise a raw foreign-key error and FastAPI would answer 500. A conflict with existing data is 409: a duplicate unique field, or a delete refused because child records remain.

Code 1.2 - schemas_db.py: unique-field format validation, and routers/routes.py: the 409 delete guard

schemas_db.py

```
ROUTE_CODE_PATTERN = r"^R-\d{4}$"          # e.g. R-0042
INCIDENT_CODE_PATTERN = r"^INC-\d{4}-\d{5}$" # e.g. INC-3170-00001
```

```
class IncidentBase(BaseModel):
    incidentCode: str = Field(
        ...,
        alias="incident_code",
        pattern=INCIDENT_CODE_PATTERN,
        description="Unique field. Format INC-NNNN-NNNNN, e.g. INC-3170-00001.",
    )
    routeId: str = Field(..., min_length=1, alias="route_id", description="Primary field")
    location: str = Field(..., min_length=1, alias="location", description="Secondary field")
    submitterEmail: EmailStr = Field(..., alias="submitter_email")
    description: str = Field(..., min_length=25)
    category: str
    ridersAffected: int = Field(
        0,
        alias="riders_affected",
        ge=0,
        description="Numeric field. Defaults to 0 when the reporter gives no estimate.",
    )
    relatedRouteId: int = Field(..., alias="related_route_id", gt=0)

    model_config = {"str_strip_whitespace": True, "populate_by_name": True}
```

```
@router.delete("/{route_id}", status_code=204)
def delete_route(route_id: int, response: Response, db: Session = Depends(get_db)):
    """Refuses with 409 while incidents still reference this route.

    No cascade is implemented: deleting a route that has incidents would
    silently destroy incident reports, which is the wrong default for an
    audit-style domain. Reassign or delete the incidents first.
    """
    _no_store(response)
    route = _get_or_404(db, route_id)
    try:
        crud.delete_route(db, route)
    except crud.RouteInUseError as exc:
        raise HTTPException(
            status_code=409,
            detail=(
                f"Route {route_id} still has {exc.incident_count} incident(s). "
                "Delete or reassign them first (no cascade is implemented).",
            ),
        )
    return Response(status_code=204)
```

[SCREENSHOT 1.3] Postman - GET /api/routes (page object: items, total, page, page_size)

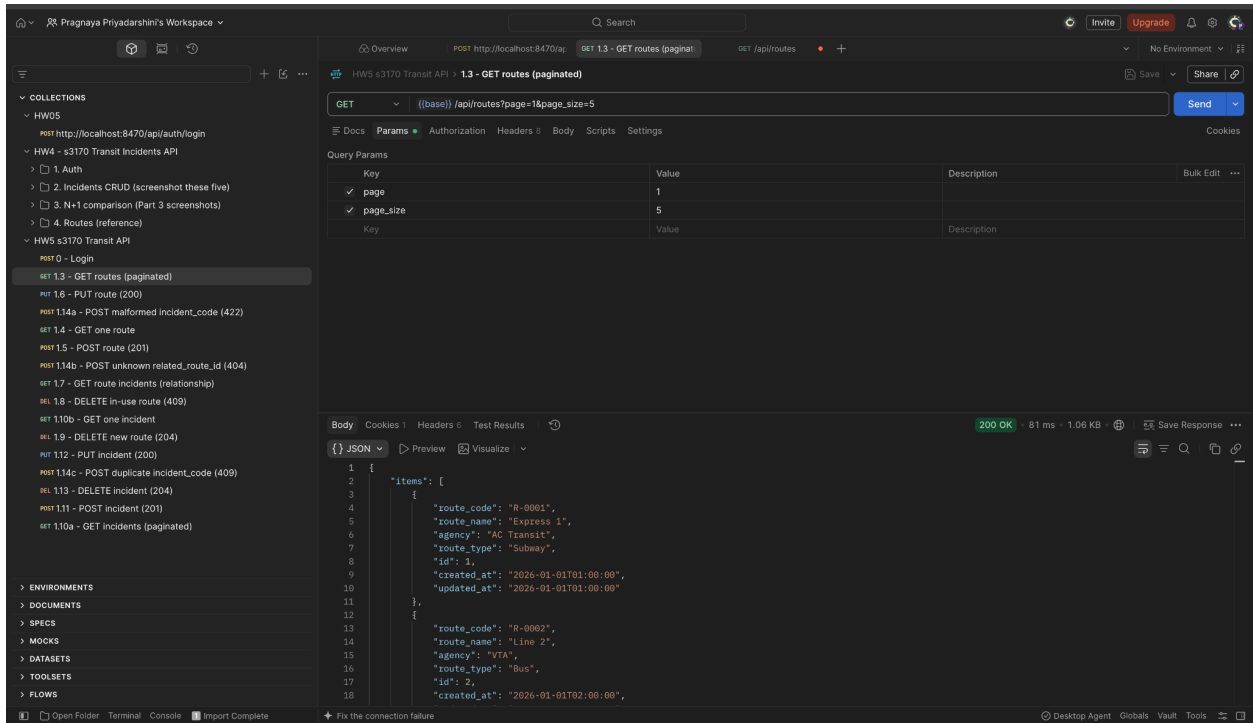


Figure 1.3 - Postman - GET /api/routes (page object: items, total, page, page_size)

[SCREENSHOT 1.4] Postman - GET /api/routes/{id} returning 200 with incident_count

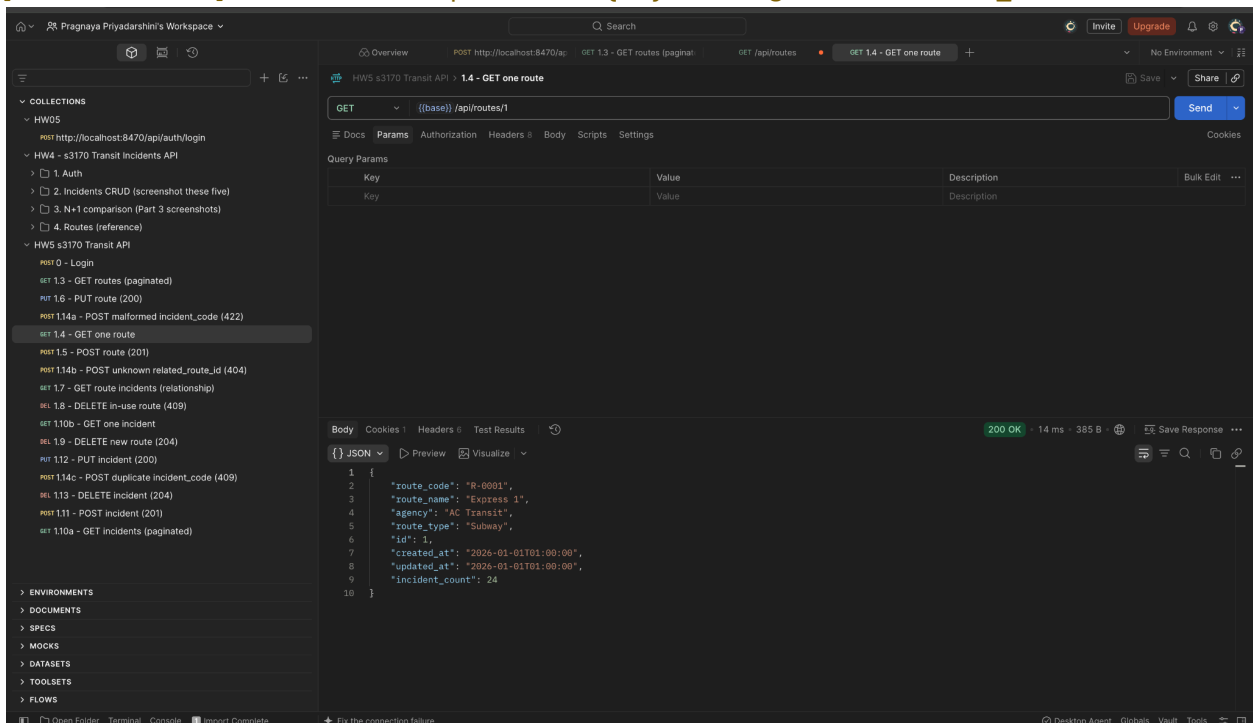


Figure 1.4 - Postman - GET /api/routes/{id} returning 200 with incident_count

[SCREENSHOT 1.5] Postman - POST /api/routes returning 201

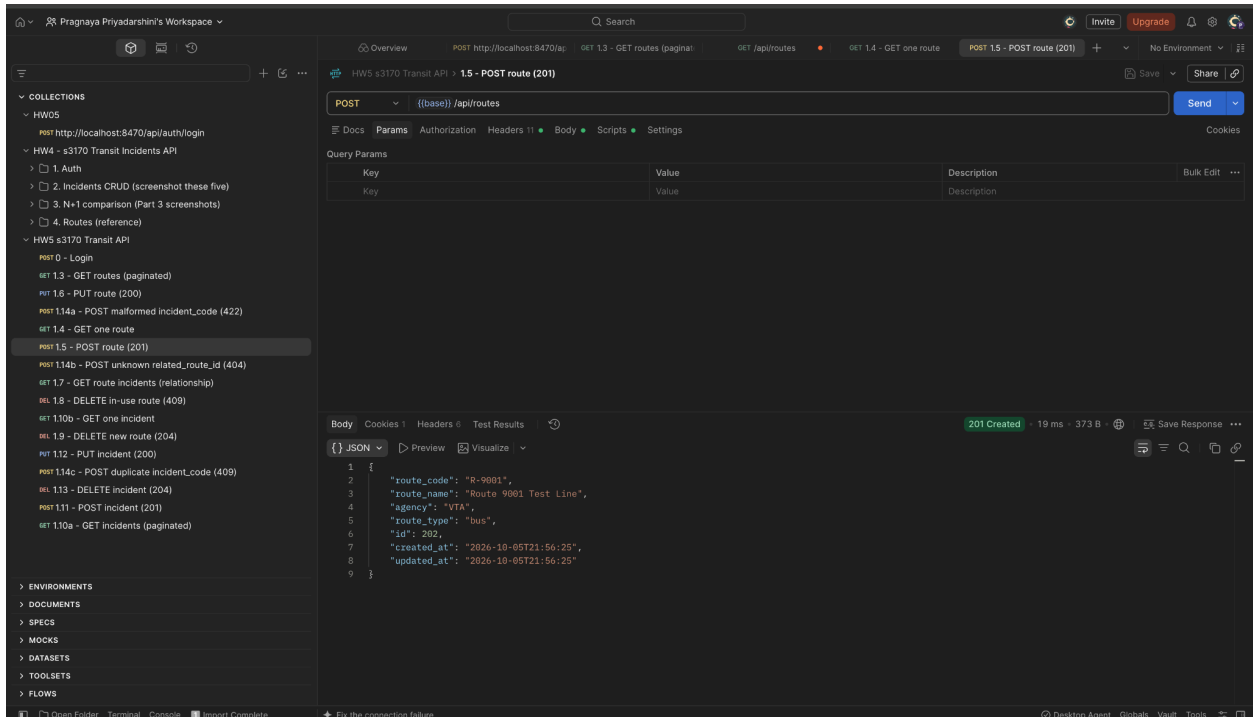


Figure 1.5 - Postman - POST /api/routes returning 201

[SCREENSHOT 1.6] Postman - PUT /api/routes/{id} returning 200

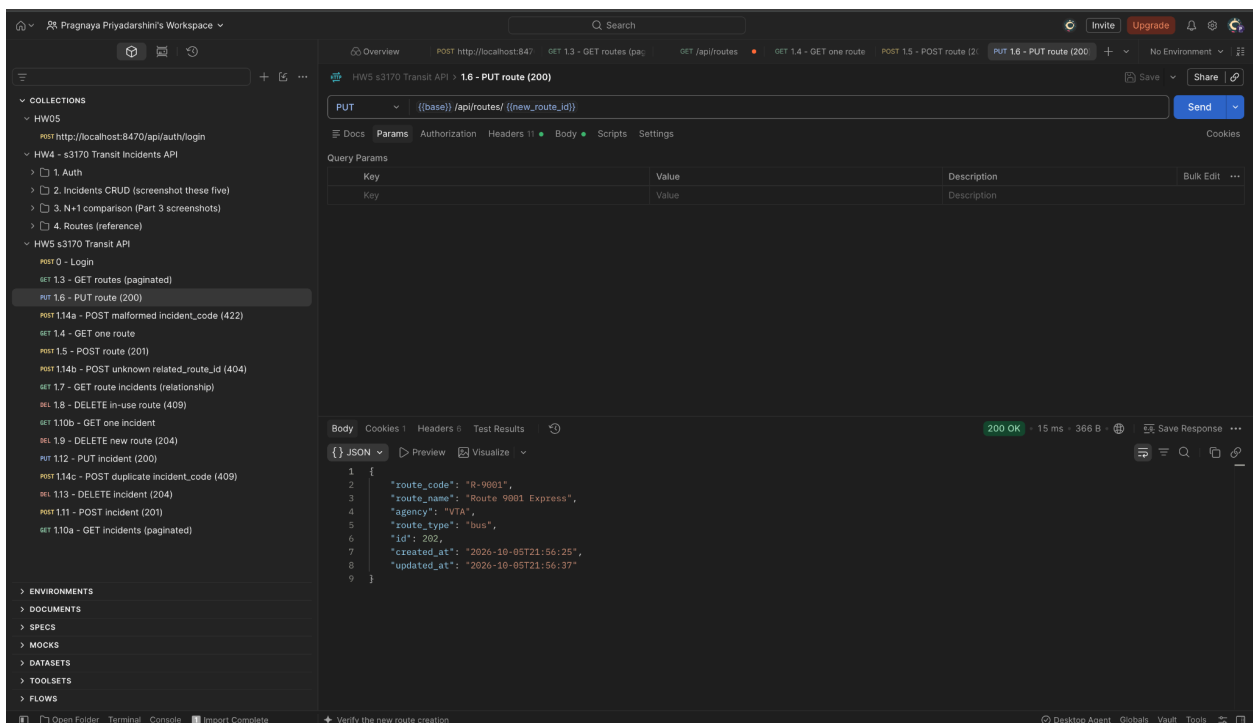


Figure 1.6 - Postman - PUT /api/routes/{id} returning 200

[SCREENSHOT 1.7] Postman - GET /api/routes/{id}/incidents, the relationship query

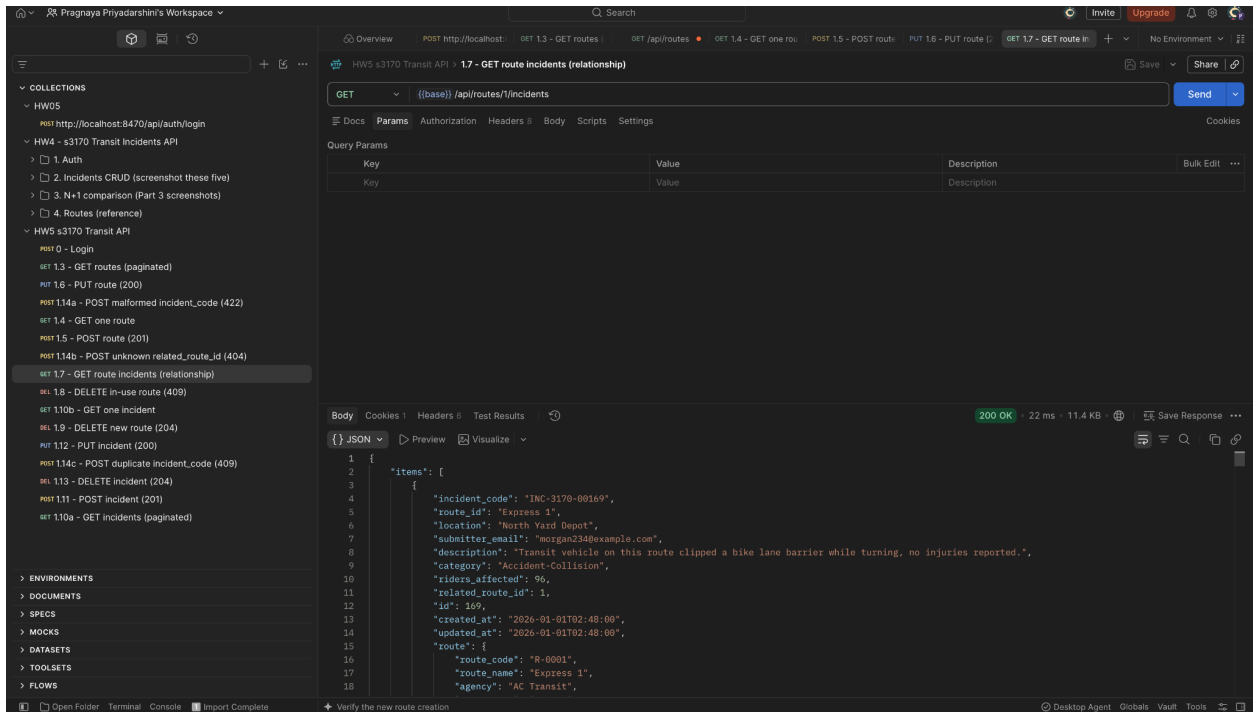


Figure 1.7 - Postman - GET /api/routes/{id}/incidents, the relationship query

[SCREENSHOT 1.8] Postman - DELETE /api/routes/{id} returning 409, with the incident count in the message

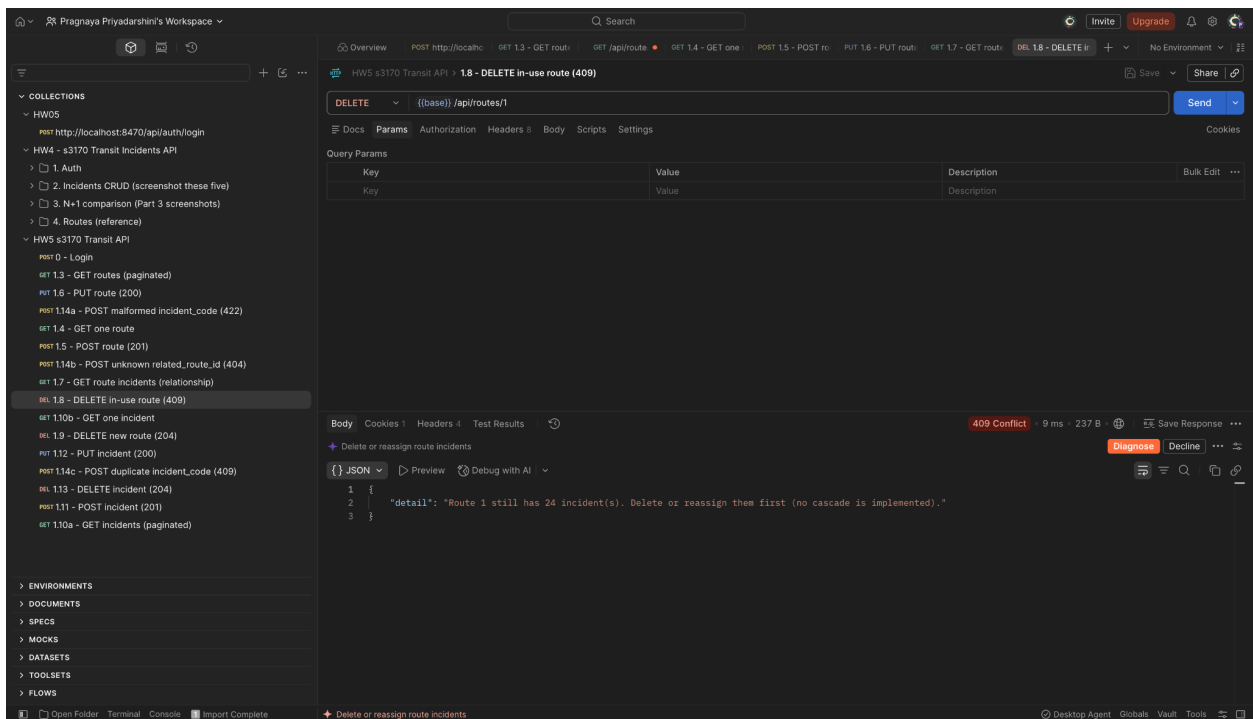


Figure 1.8 - Postman - DELETE /api/routes/{id} returning 409, with the incident count in the message

[SCREENSHOT 1.9] Postman - DELETE /api/routes/{id} returning 204 (the route created in 1.5, which has no incidents)

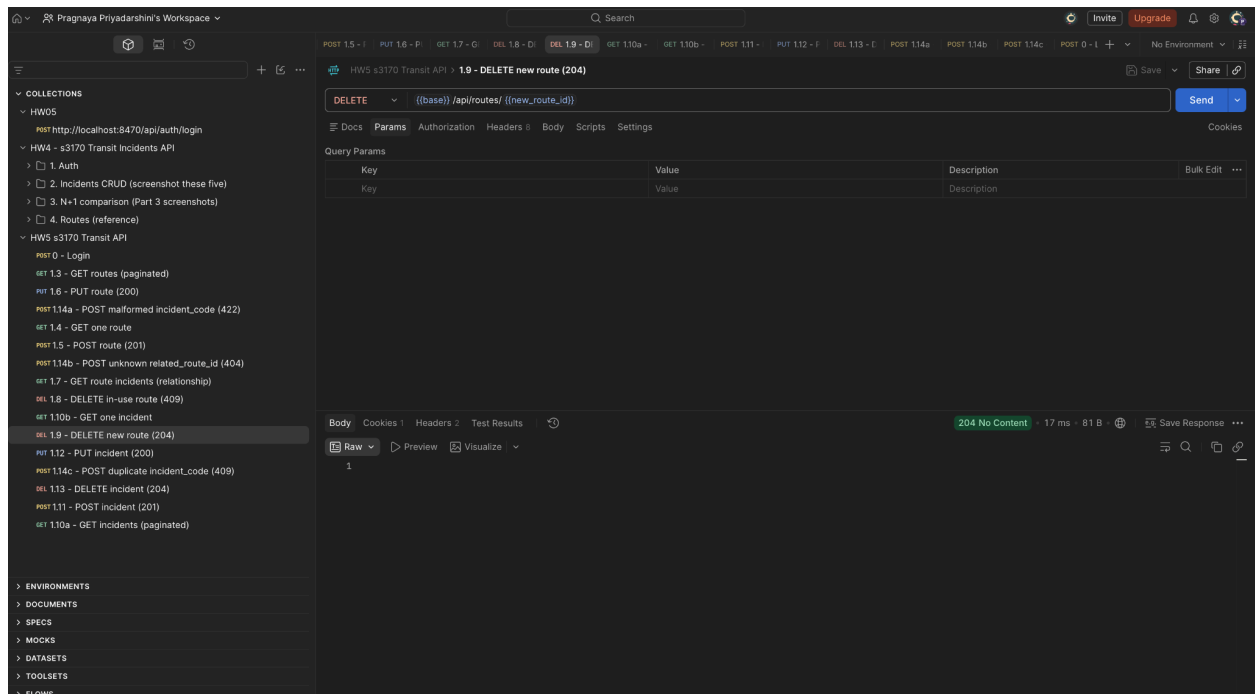
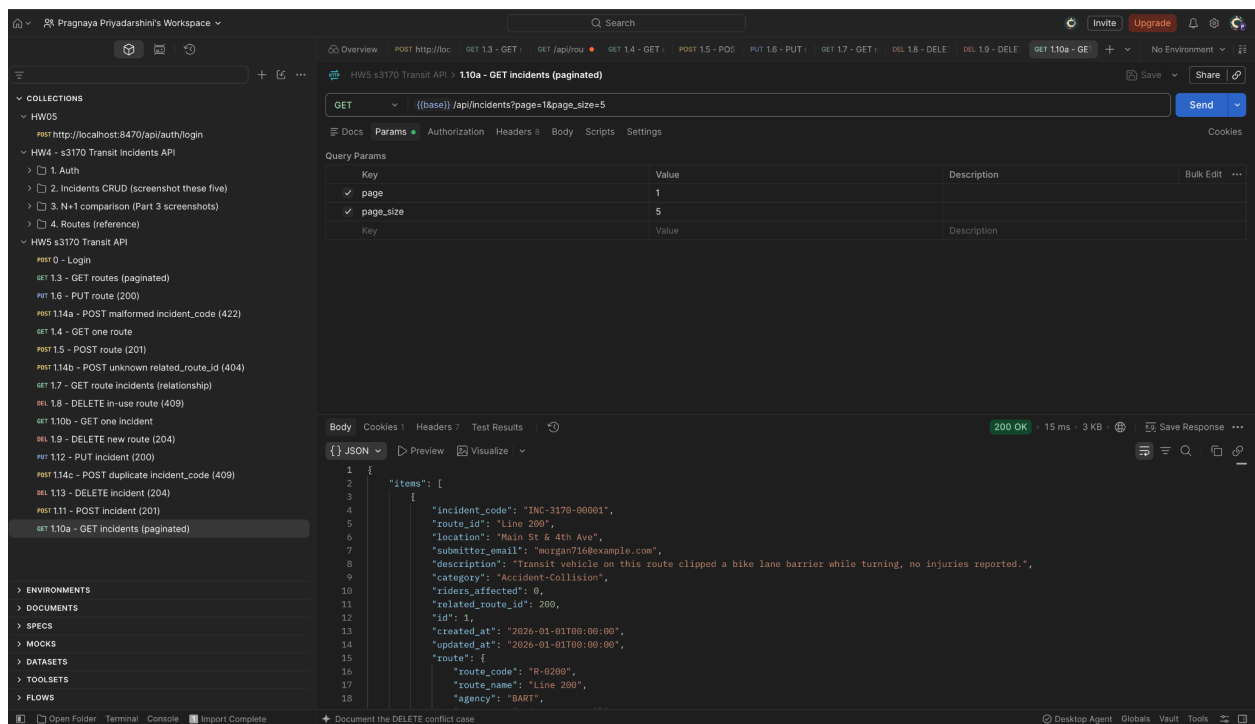


Figure 1.9 - Postman - DELETE /api/routes/{id} returning 204 (the route created in 1.5, which has no incidents)

[SCREENSHOT 1.10] Postman - GET /api/incidents and GET /api/incidents/{id}



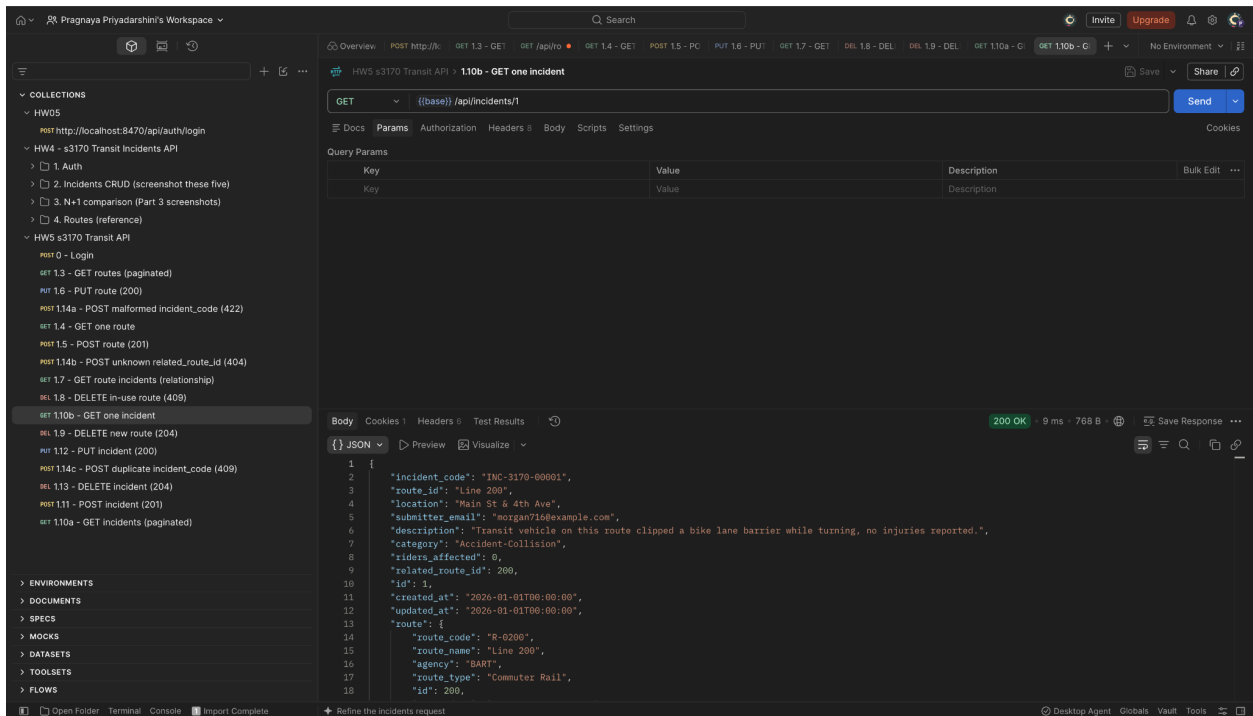


Figure 1.10 - Postman - GET /api/incidents and GET /api/incidents/{id}

[SCREENSHOT 1.11] Postman - POST /api/incidents returning 201

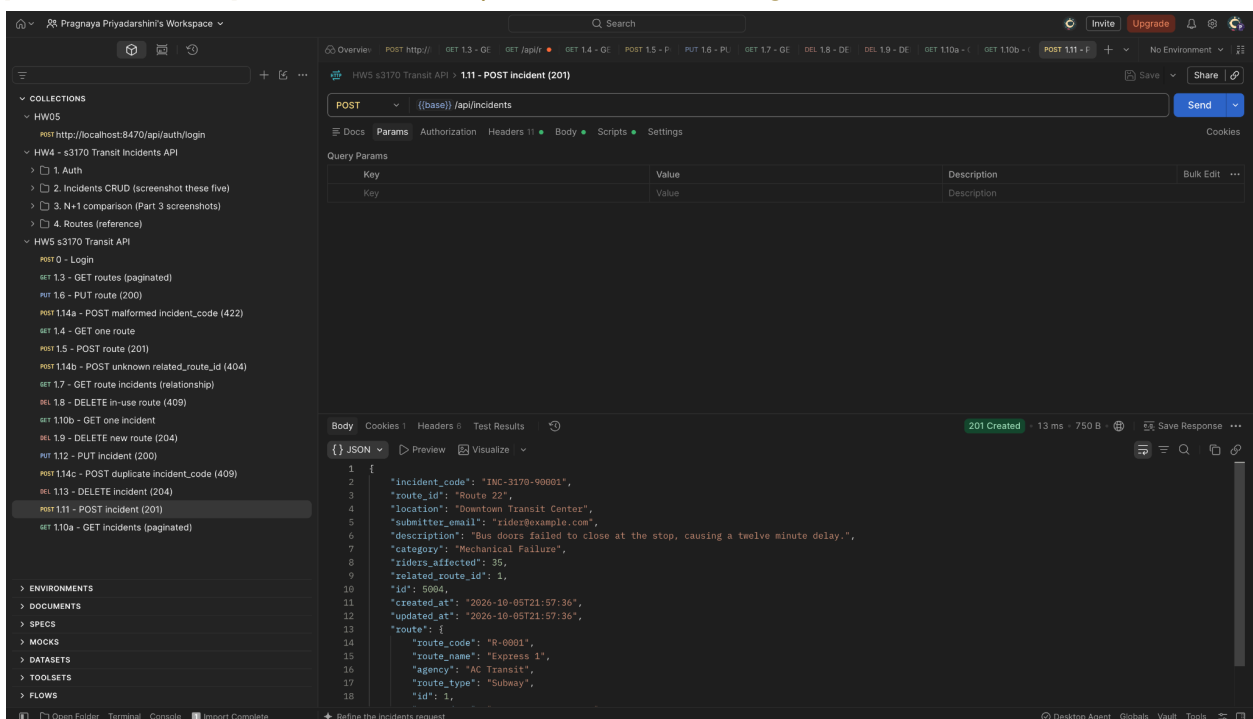


Figure 1.11 - Postman - POST /api/incidents returning 201

[SCREENSHOT 1.12] Postman - PUT /api/incidents/{id} returning 200

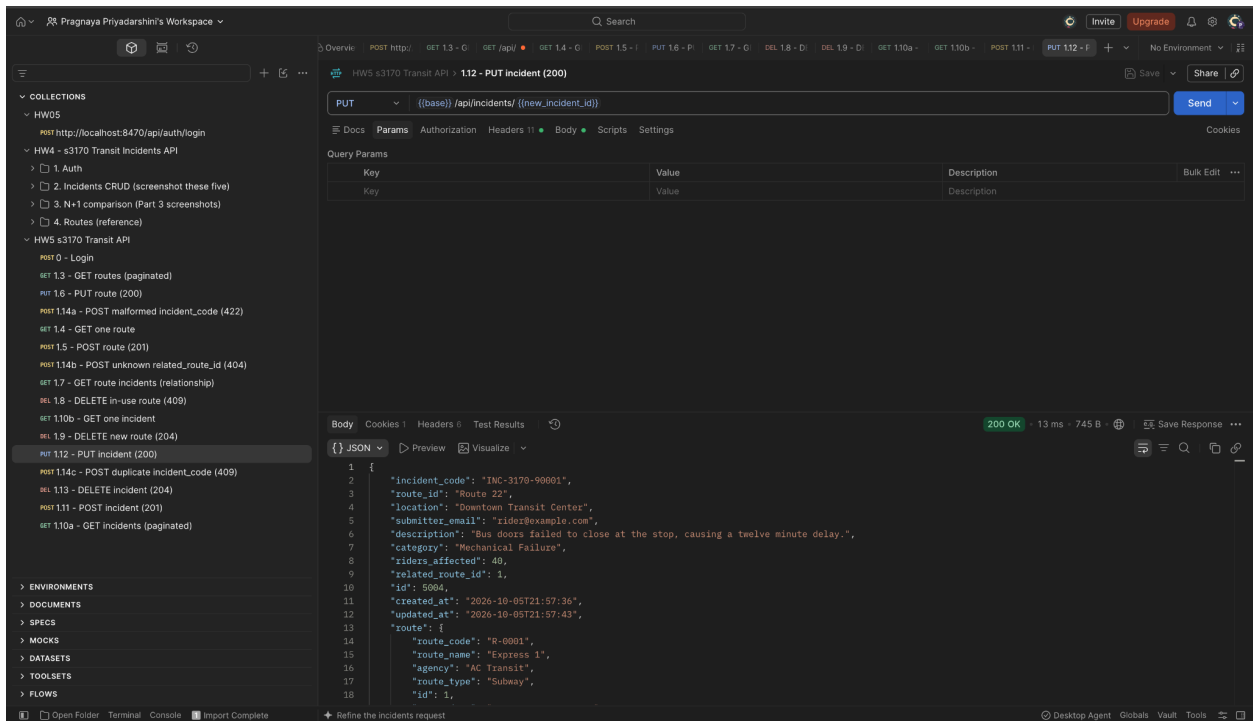


Figure 1.12 - Postman - PUT `/api/incidents/{id}` returning 200

[SCREENSHOT 1.13] Postman - DELETE `/api/incidents/{id}` returning 204

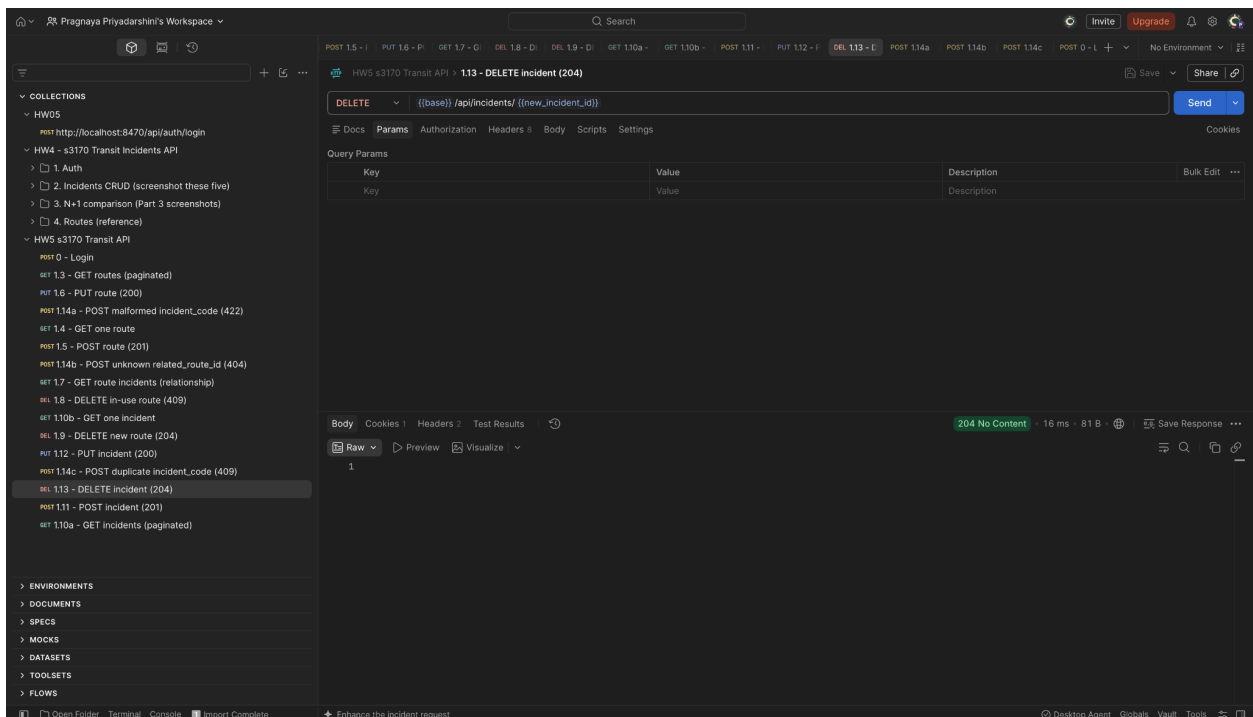


Figure 1.13 - Postman - DELETE `/api/incidents/{id}` returning 204

[SCREENSHOT 1.14] Postman - error cases: 422 (malformed `incident_code`), 404 (unknown `related_route_id`), 409 (duplicate `incident_code`)

Pragnaya Priyadarshini's Workspace

Search

Invite Upgrade

HT 1.3 - G GET /api GET 1.4 - G POST 1.5 - I PUT 1.6 - P GET 1.7 - G DEL 1.8 - D DEL 1.9 - D GET 1.10a - GET 1.10b - POST 1.11 - PUT 1.12 - I DEL 1.13 - E POST 1.14a + No Environment

HW5 s3170 Transit API 1.14a - POST malformed incident_code (422)

POST (base) /api/incidents Send

Docs Params Authorization Headers 1 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies 1 Headers 4 Test Results 422 Unprocessable Entity (WebDAV) (RFC 4918) 20 ms · 407 B Save Response

+ Fix incident_code pattern

```
1 {
2   "detail": [
3     {
4       "type": "string_pattern_mismatch",
5       "loc": [
6         "body",
7         "incident_code"
8       ],
9       "msg": "String should match pattern '^INC-\\d{4}-\\d{5}$'",
10      "input": "INC-317-1",
11      "ctx": {
12        "pattern": "^INC-\\d{4}-\\d{5}$"
13      },
14      "url": "https://errors.pydantic.dev/2.12/v/string_pattern_mismatch"
15    }
16  ]
17 }
```

Open Folder Terminal Console Import Complete Desktop Agent Globals Vault Tools

Pragnaya Priyadarshini's Workspace

Search

Invite Upgrade

HT /api GET 1.4 - G POST 1.5 - I PUT 1.6 - P GET 1.7 - G DEL 1.8 - D DEL 1.9 - D GET 1.10a - GET 1.10b - POST 1.11 - PUT 1.12 - I DEL 1.13 - E POST 1.14a POST 1.14b + No Environment

HW5 s3170 Transit API 1.14b - POST unknown related_route_id (404)

POST (base) /api/incidents Send

Docs Params Authorization Headers 1 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies 1 Headers 4 Test Results 404 Not Found 17 ms · 166 B Save Response

+ Continue in Agent Mode Diagnose Decline

```
1 {
2   "detail": "Route 99999 not found"
3 }
```

Open Folder Terminal Console Import Complete Desktop Agent Globals Vault Tools

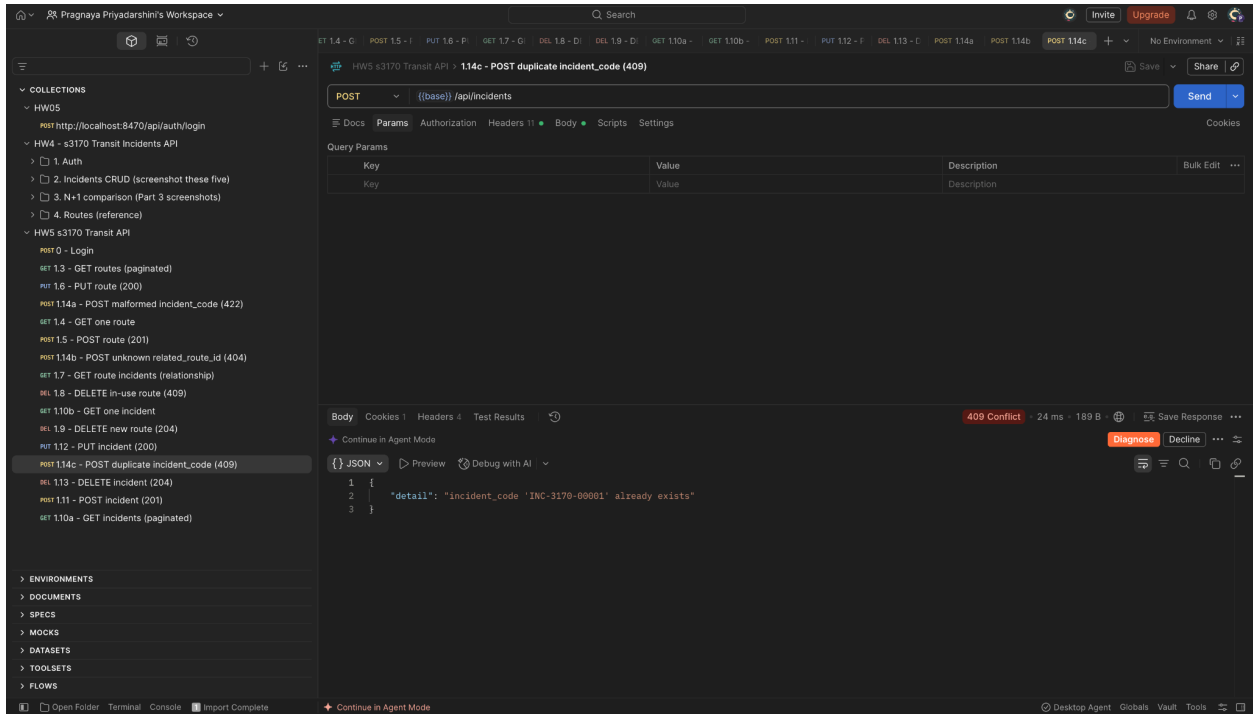
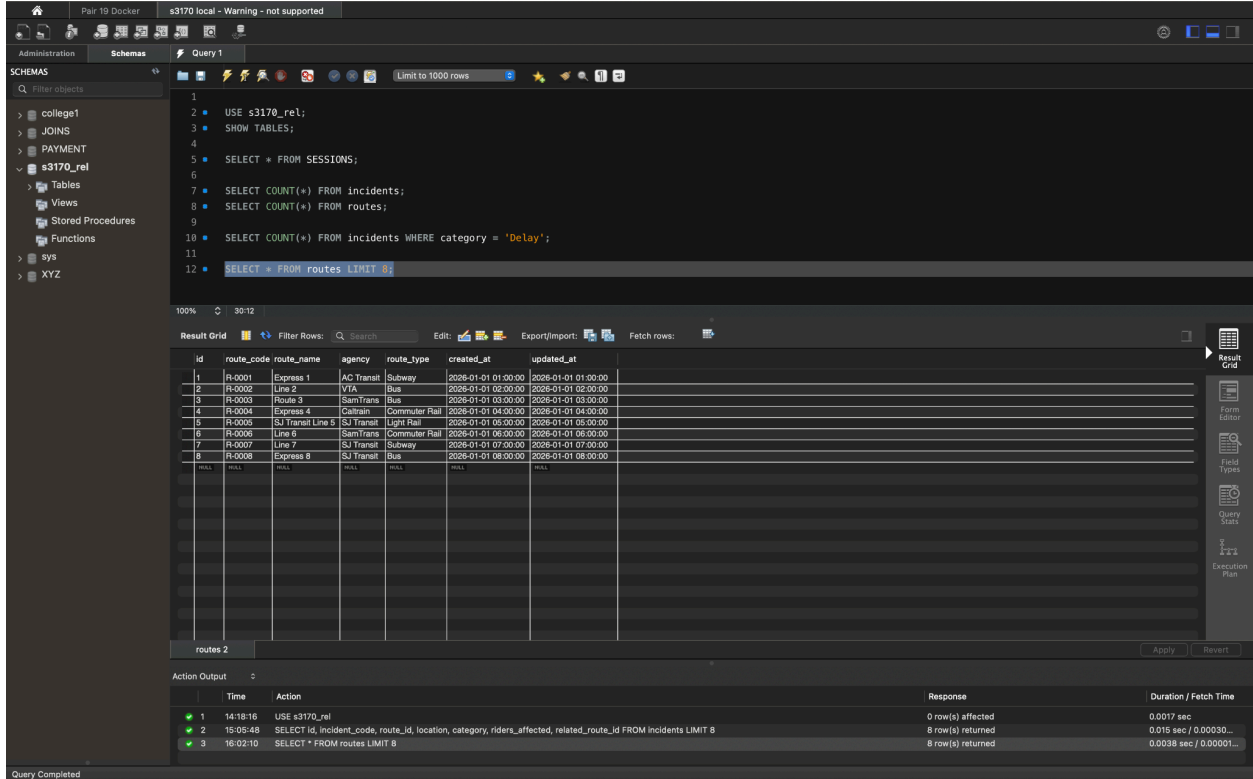


Figure 1.14 - Postman - error cases: 422 (malformed incident_code), 404 (unknown related_route_id), 409 (duplicate incident_code)

[SCREENSHOT 1.15] Database - the incidents and routes tables with rows visible



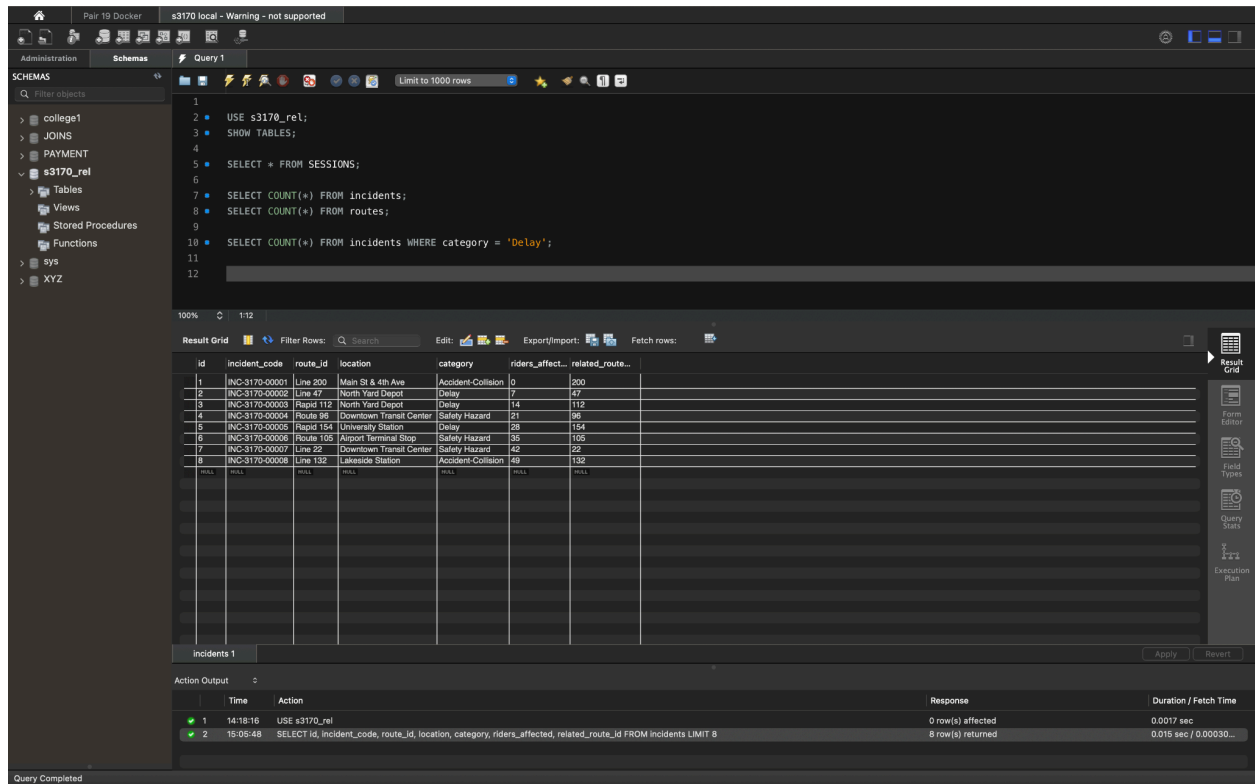


Figure 1.15 - Database - the incidents and routes tables with rows visible

III. Redux Client

The React data layer moved from component state to Redux Toolkit. Through HW4, App.jsx held five useState hooks, a refresh callback and five handler functions, and passed them down as props to every screen. In HW5 a configureStore store (src/store/index.js) holds that state, main.jsx wraps the app in <Provider>, and each screen reads the store directly with useSelector, so App.jsx keeps only routing.

The incidents slice (src/store/incidentsSlice.js) has four async thunks that call the FastAPI endpoints through axios: fetchIncidents (GET), createIncident (POST), updateIncident (PUT /{id}) and deleteIncident (DELETE /{id}). Each generates pending, fulfilled and rejected actions, handled in extraReducers.

Success and error handling. Thunks use rejectWithValue to surface the API's detail string, so a 409 reaches the user as "incident_code 'INC-3170-00001' already exists" rather than "Request failed with status code 409". On success the reducers update state in place - createIncident pushes the returned record, updateIncident replaces it by findIndex, deleteIncident filters it out - so the Home list updates without a second request.

[SCREENSHOT 1.16] Home - the fetchIncidents thunk and its three extraReducers cases, beside the populated table

Code and UI

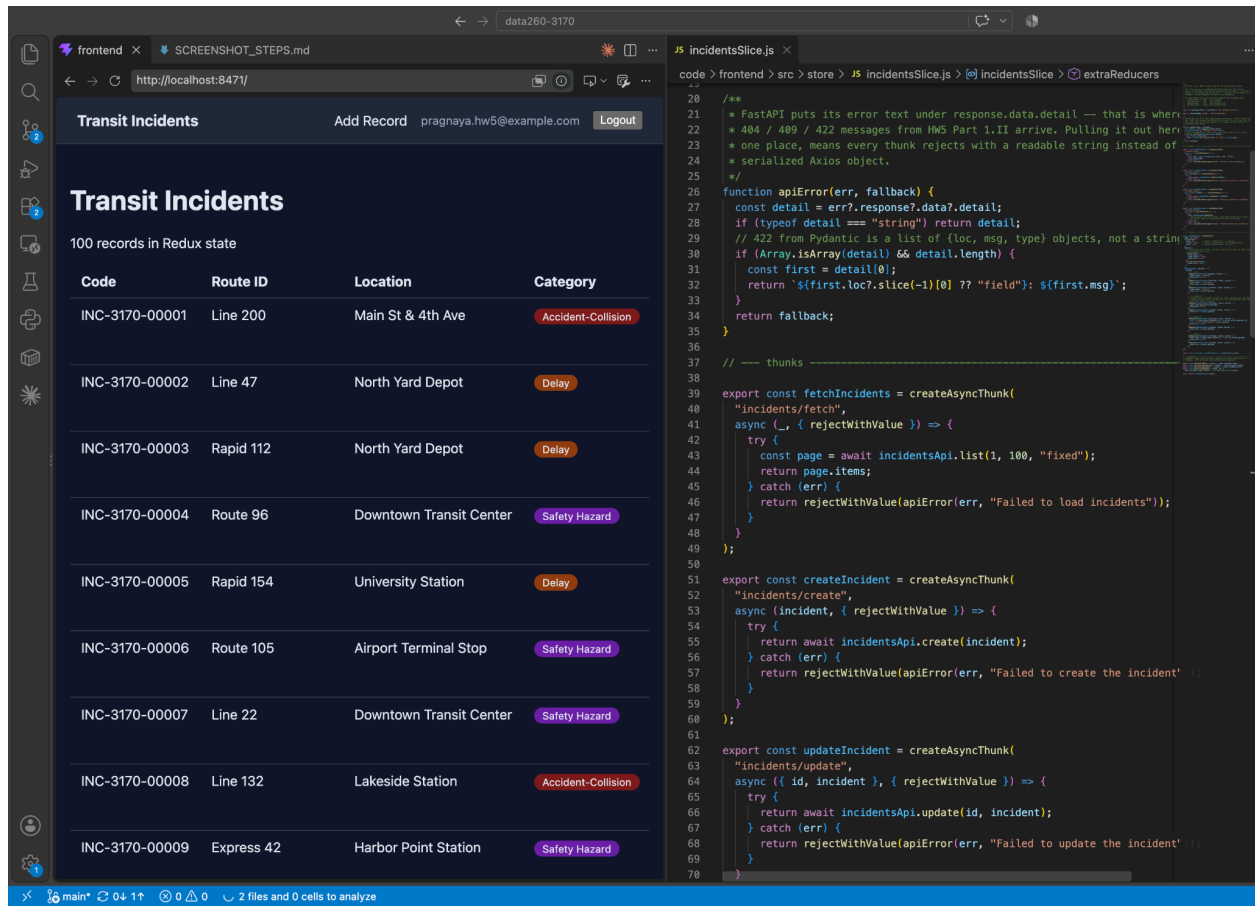


Figure 1.16 - Home - the `fetchIncidents` thunk and its three `extraReducers` cases, beside the populated table

[SCREENSHOT 1.17] Create - the `createIncident` thunk and its fulfilled case, beside the filled form and the new row on Home

Code	Route ID	Location	Category	Riders	Description
INC-3170-00001	Line 200	Main St & 4th Ave	Accident-Collision	0	Transit vehicle on this i clipped a bike lane ban turning, no injuries rep
INC-3170-00002	Line 47	North Yard Depot	Delay	7	Transit vehicle on this i running 54 minutes bel schedule with no drive
INC-3170-00003	Rapid 112	North Yard Depot	Delay	14	Transit vehicle on this i running 16 minutes bel schedule with no drive
INC-3170-00004	Route 96	Downtown Transit Center	Safety Hazard	21	Transit vehicle on this i a slippery floor from a l no warning signage po
INC-3170-00005	Rapid 154	University Station	Delay	28	Transit vehicle on this i skipped the stop entire stranding riders for 12
INC-3170-00006	Route 105	Airport Terminal Stop	Safety Hazard	35	Transit vehicle on this i exposed wiring near th door reported by a ride
INC-3170-00007	Line 22	Downtown Transit Center	Safety Hazard	42	Transit vehicle on this i a broken step that ride tripped over boarding.
INC-3170-00008	Line 132	Lakeside Station	Accident-Collision	49	Transit vehicle on this i clipped a bike lane ban turning, no injuries rep
INC-3170-00009	Express 42	Harbor Point Station	Safety Hazard	56	Transit vehicle on this i a broken step that ride tripped over boarding.
INC-3170-00010	SJ Transit Line 24	Civic Center Station	Accident-Collision	63	Transit vehicle on this i clipped a bike lane ban turning, no injuries rep
INC-3170-00011	Express 190	Central Square	Delay	70	Transit vehicle on this i never arrived and rider 31 minutes with no

Figure 1.17 - Create - the createIncident thunk and its fulfilled case, beside the filled form and the new row on Home

[SCREENSHOT 1.18] Update - the updateIncident thunk and its findIndex replacement, beside the edit form and the changed row

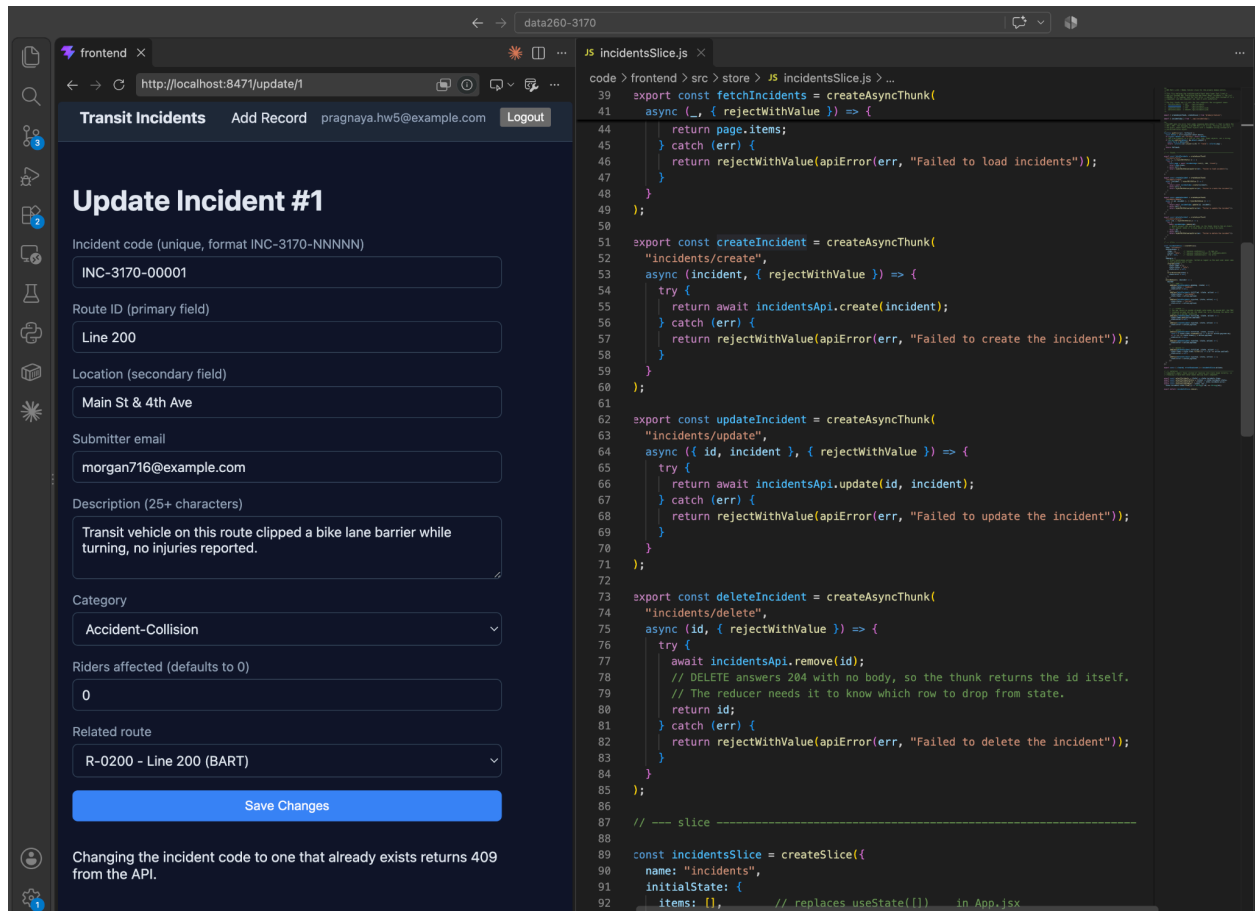


Figure 1.18 - Update - the updateIncident thunk and its findIndex replacement, beside the edit form and the changed row

[SCREENSHOT 1.19] Delete - the deleteIncident thunk and its filter, beside the delete button and the row removed

before deleting:

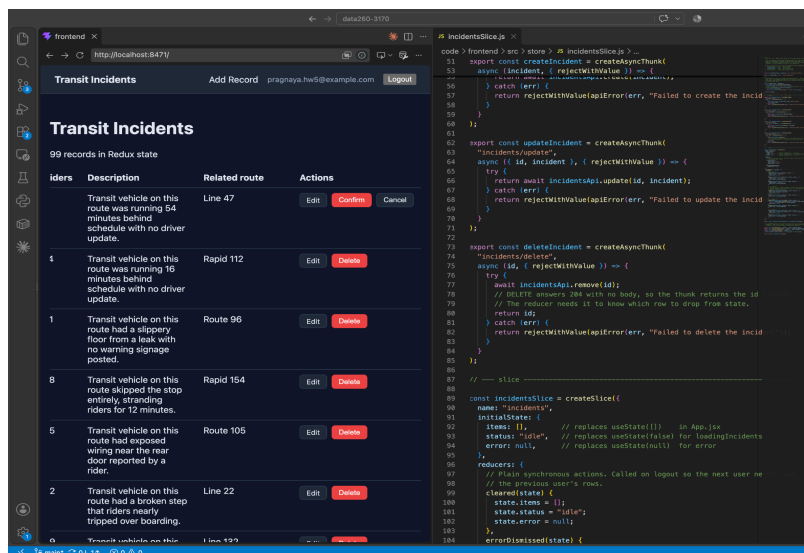
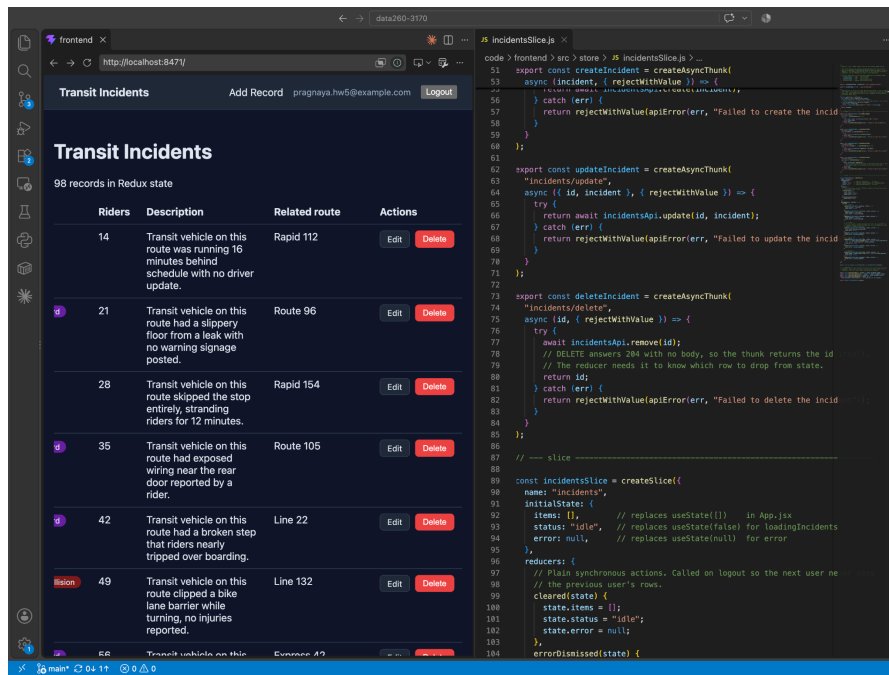


Figure 1.19 - Delete - the deleteIncident thunk and its filter, beside the delete button and the row removed

After deleting:



Part 2 - MCP Tool Servers

Two local MCP servers, both FastMCP over STDIO. stdout carries the JSON-RPC stream, so writing to it corrupts the stream and the Inspector disconnects. Both servers configure logging to stderr before any other import and contain no `print()` calls; the verification script checks this mechanically.

Both were run and debugged in the MCP Inspector, launched with `npx @modelcontextprotocol/inspector .venv/bin/python <server>.py`. This is the Inspector that `mcp dev` wraps; `mcp dev` itself needs `uv`, and launching through the project's virtualenv keeps the transit server's MySQL driver available.

A. TheMealDB Tutorial Server

A server named `meals` exposing four tools over TheMealDB's public JSON API with the published test key

1. Network and JSON failures raise a clean error that the Inspector displays; a "no results" response (`{"meals": null}`) is not a failure and returns an empty result with a short "no matches" message.

Tool	Endpoint	Returns
<code>search_meals_by_name(query, limit=5)</code>	<code>search.php?s=</code>	id, name, area, category, thumb (limit 1-25)
<code>meals_by_ingredient(ingredient, limit=12)</code>	<code>filter.php?i=</code>	id, name, thumb
<code>random_meal()</code>	<code>random.php</code>	full recipe (same shape as <code>meal_details</code>)
<code>meal_details(id)</code>	<code>lookup.php?i=</code>	id, name, category, area, instructions, image, source, youtube, ingredients[{name, measure}]

Table 2.1 - the four tutorial tools. TheMealDB stores ingredients as twenty flat `strIngredient/strMeasure` pairs; the server zips them into a list and drops the blanks.

Code 2.1 - `meals_server.py`: one tool, showing the input clamp and the "no matches" path

[SCREENSHOT 2.1] MCP Inspector - `search_meals_by_name`, `query="Arrabiata"`, showing input and output

```
@mcp.tool()
def search_meals_by_name(query: str, limit: int = 5) -> dict[str, Any]:
    """Search meals by name.

    Args:
        query: the dish name to search for, e.g. "Arrabiata".
        limit: how many meals to return, 1-25.

    Returns {query, count, meals[], message}. Each meal carries id, name,
    area, category and thumb; use the id with meal_details for the full recipe.
    """

    if not query or not query.strip():
        return {"query": query, "count": 0, "meals": [], "message": "query is required"}
    limit = max(1, min(int(limit), 25))

    payload = _get("search.php", {"s": query.strip()})
    meals = payload.get("meals")
    if not meals:
        log.info("search_meals_by_name %r -> no matches", query)
        return {
            "query": query,
            "count": 0,
            "meals": [],
            "message": f"no matches for {query!r}",
        }

    cards = [_card(m) for m in meals[:limit]]
    log.info("search_meals_by_name %r -> %d of %d", query, len(cards), len(meals))
    return {"query": query, "count": len(cards), "meals": cards, "message": None}
```

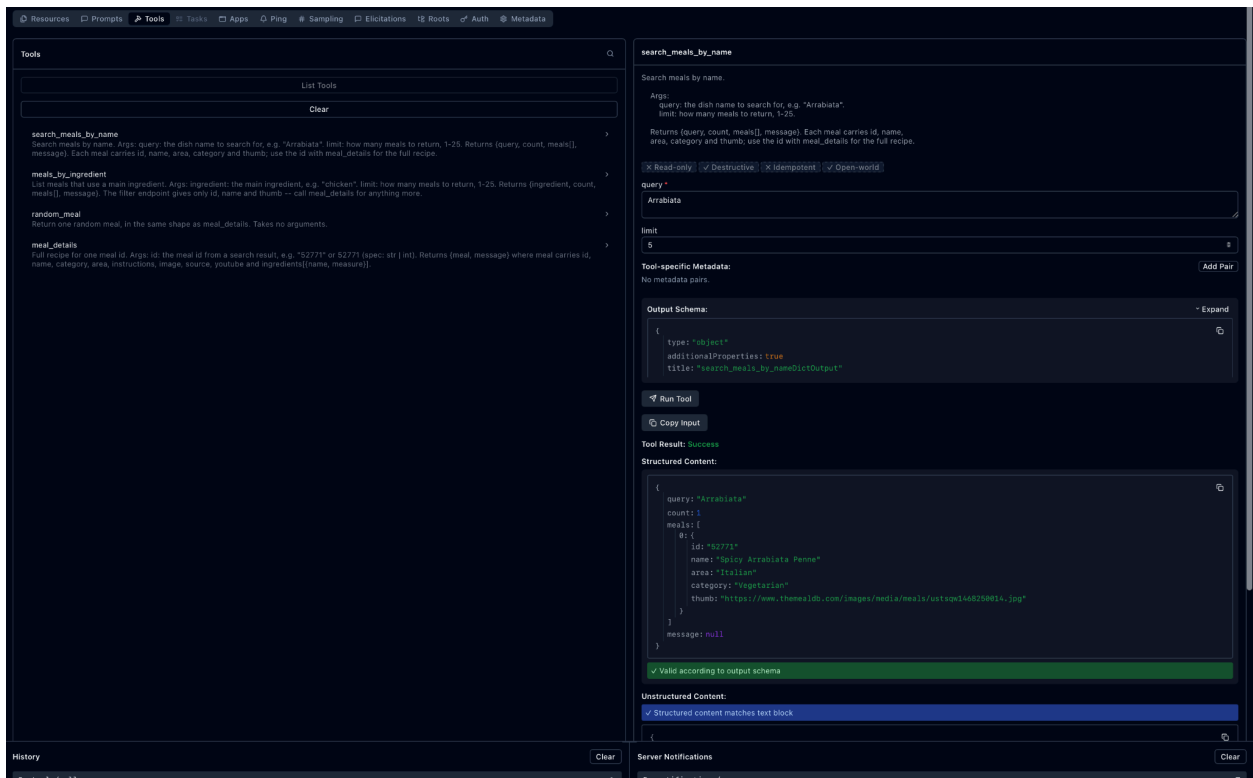


Figure 2.1 - MCP Inspector - search_meals_by_name, query="Arrabiata", showing input and output

[SCREENSHOT 2.2] MCP Inspector - meals_by_ingredient, ingredient="chicken"

```

@mcp.tool()
def meals_by_ingredient(ingredient: str, limit: int = 12) -> dict[str, Any]:
    """List meals that use a main ingredient.

    Args:
        ingredient: the main ingredient, e.g. "chicken".
        limit: how many meals to return, 1-25.

    Returns {ingredient, count, meals[], message}. The filter endpoint gives
    only id, name and thumb -- call meal_details for anything more.
    """
    if not ingredient or not ingredient.strip():
        return {
            "ingredient": ingredient,
            "count": 0,
            "meals": [],
            "message": "ingredient is required",
        }
    limit = max(1, min(int(limit), 25))

    payload = _get("filter.php", {"i": ingredient.strip()})
    meals = payload.get("meals")
    if not meals:
        log.info("meals_by_ingredient %r -> no matches", ingredient)
        return {
            "ingredient": ingredient,
            "count": 0,
            "meals": [],
            "message": f"no matches for {ingredient!r}",
        }

    cards = [_small_card(m) for m in meals[:limit]]
    log.info("meals_by_ingredient %r -> %d of %d", ingredient, len(cards), len(meals))
    return {
        "ingredient": ingredient,
        "count": len(cards),
        "meals": cards,
        "message": None,
    }

```

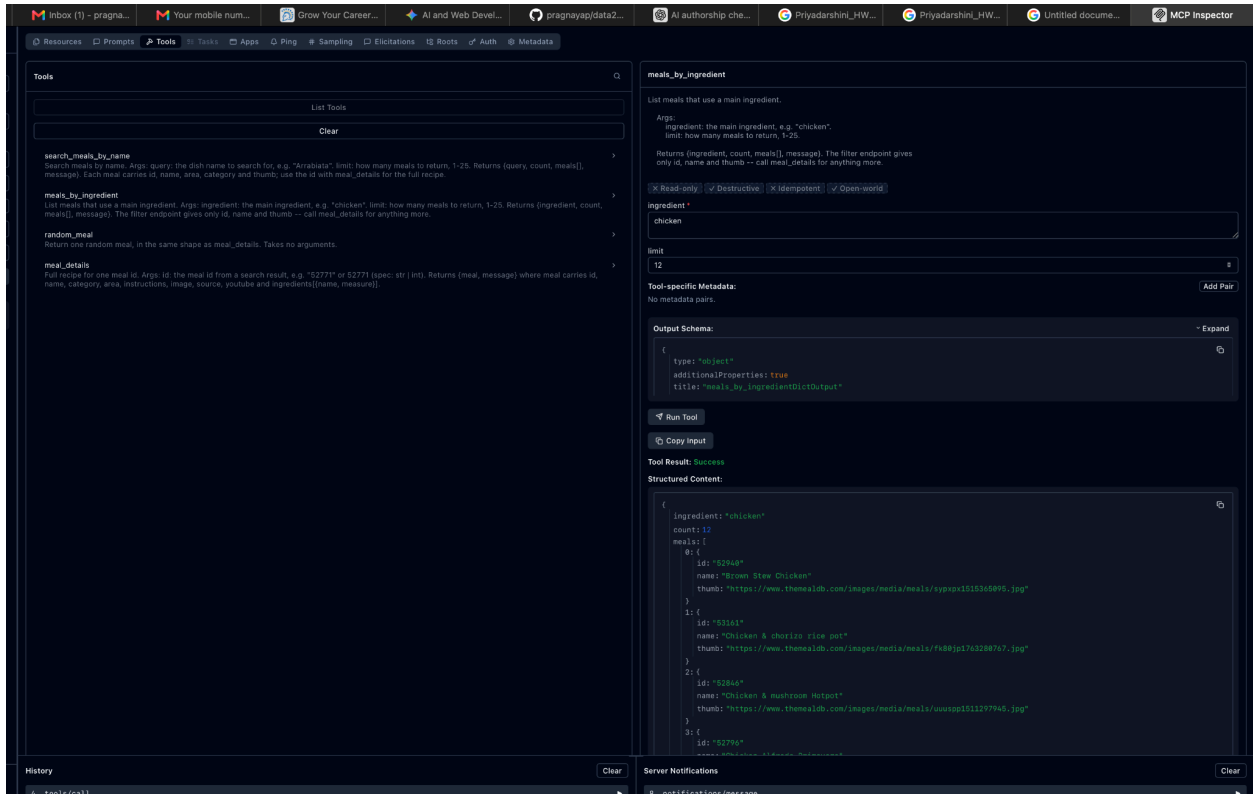
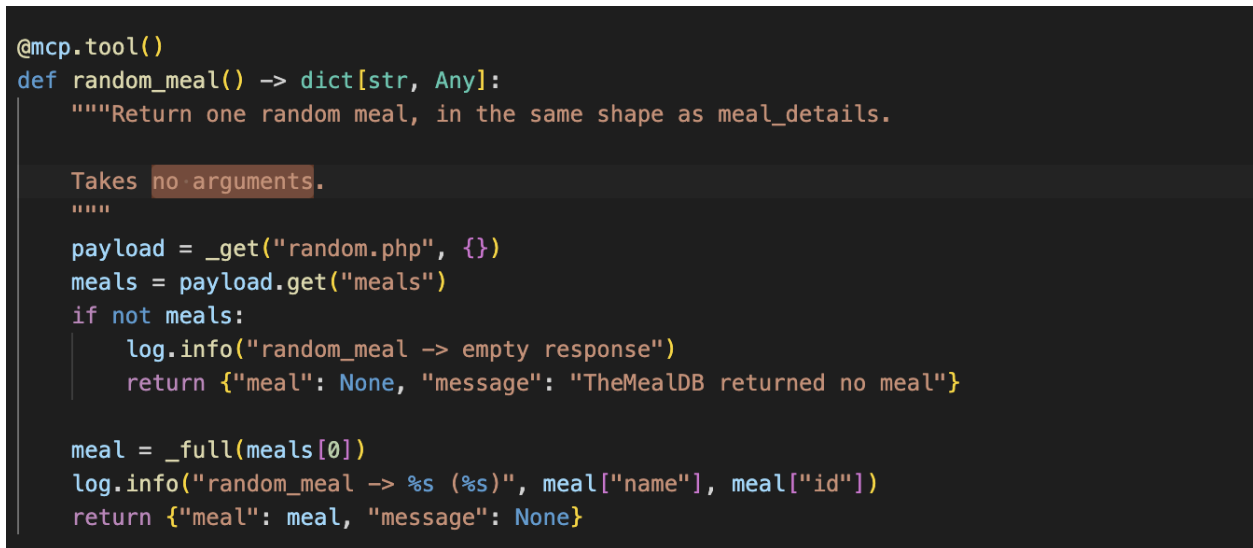


Figure 2.2 - MCP Inspector - meals_by_ingredient, ingredient="chicken"

[SCREENSHOT 2.3] MCP Inspector - random_meal, no arguments



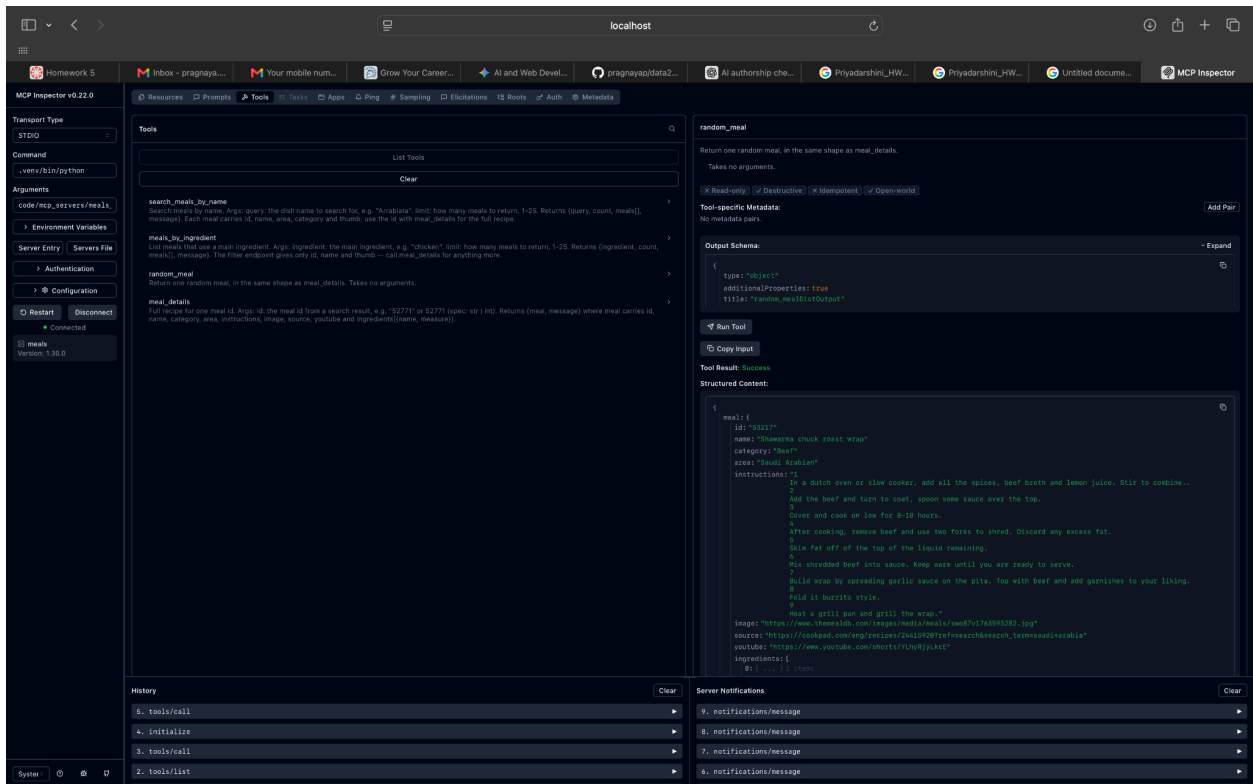


Figure 2.3 - MCP Inspector - random_meal, no arguments

[SCREENSHOT 2.4] MCP Inspector - meal_details, id="52771"

```
@mcp.tool()
def meal_details(id: str) -> dict[str, Any]:
    """Full recipe for one meal id.

    Args:
        id: the meal id from a search result, e.g. "52771".

    Returns {meal, message} where meal carries id, name, category, area,
    instructions, image, source, youtube and ingredients[{name, measure}].
    """
    meal_id = str(id).strip()
    if not meal_id:
        return {"meal": None, "message": "id is required"}

    payload = _get("lookup.php", {"i": meal_id})
    meals = payload.get("meals")
    if not meals:
        log.info("meal_details %r -> no match", meal_id)
        return {"meal": None, "message": f"no meal found with id {meal_id!r}"}

    meal = _full(meals[0])
    log.info("meal_details %s -> %s", meal_id, meal["name"])
    return {"meal": meal, "message": None}

if __name__ == "__main__":
    log.info("meals MCP server starting on STDIO (4 tools)")
    mcp.run()
```

The screenshot displays the MCP Inspector interface, which is used to manage and interact with MCP tools. The interface is divided into several sections:

- Tools List:** A table on the left lists available tools:
 - search_meals_by_name:** Search meals by name. Args: query: the dish name to search for, e.g. "Arabiata". limit: how many meals to return, 1-25. Returns (query, count, meals[]).
 - meals_by_ingredient:** List meals that use a main ingredient. Args: ingredient: the main ingredient, e.g. "chicken". limit: how many meals to return, 1-25. Returns (ingredient, count, meals[]).
 - random_meal:** Return one random meal, in the same shape as meal_details. Takes no arguments.
 - meal_details:** Full recipe for one meal id. Args: id: the meal id from a search result, e.g. "52771" or 52771 (spec: str | int). Returns (meal, message) where meal carries id, name, category, area, instructions, image, source, youtube and ingredients[{name, measure}].
- meal_details Configuration:** On the right, the configuration for the 'meal_details' tool is shown. It includes the description, arguments (id), and a list of flags (Read-only, Destructive, Idempotent, Open-world). The 'id' argument is set to '52771'.
- Tool-specific Metadata:** A section below the configuration showing 'No metadata pairs'.
- Output Schema:** A section showing the JSON schema for the tool's output:


```
{
  type: "object"
  additionalProperties: true
  title: "meal_detailsOutput"
}
```
- Run Tool:** A button to execute the tool with the current configuration.
- Copy Input:** A button to copy the input arguments.
- Tool Result:** A section showing the result of the tool execution:


```
Success
Structured Content:
{
  meal: {
    id: "52771"
    name: "Easy Arrabiata Penne"
    category: "Vegetarian"
    area: "Italian"
    instructions: "Bring a large pot of water to a boil. Add kosher salt to the boiling water, then add the pasta. Cook according to the package instructions, about 9 minutes. In a large skillet over medium-high heat, add the olive oil and heat until the oil starts to shimmer. Add the garlic and cook, stirring, until fragrant, 1 to 2 minutes. Add the chopped tomatoes, red chili flakes, Italian seasoning and salt and pepper to taste. Bring to a boil and cook for 5 minutes. Remove from the heat and add the chopped basil. Drain the pasta and add it to the sauce. Garnish with Parmigiano-Reggiano flakes and more basil and serve with more."
    image: "https://www.themebdb.com/images/media/meals/ustqps44825891a.jpg"
    source: null
    youtube: "https://www.youtube.com/watch?v=5lss7_gu188"
  }
}
```
- History:** A section at the bottom left showing a list of recent tool calls:
 - 6. tools/call
 - 5. tools/call
 - 4. initialize
 - 3. tools/call
- Server Notifications:** A section at the bottom right showing a list of notifications:
 - 12. notifications/message
 - 11. notifications/message
 - 10. notifications/message
 - 9. notifications/message

Figure 2.4 - MCP Inspector - meal_details, id="52771"

B. Domain MCP Server

A second server (s3170-transit) over the s3170_re1 data with exactly three tools. All three return one response envelope, {ok, data, error}, with error null on success. The envelope is defined once in code/mcp_servers/envelope.py and imported by every later part: Part 3 reads .ok to score fault injection, Part 4's execute_tool serializes this exact shape, and Part 5's safety rule returns it instead of raising.

The tool logic lives in transit_tools.py and the MCP wiring in transit_server.py. The split lets Part 4's offline tests call the three functions directly, with no MCP process and no network.

Code 2.2 - envelope.py: the one response shape used by Parts 2B-5

```
from __future__ import annotations

from typing import Any, TypedDict

class Envelope(TypedDict):
    """The one response shape. Every tool in Parts 2B-5 returns this."""
    ok: bool
    data: Any | None
    error: str | None

def ok(data: Any) -> Envelope:
    """A successful call. `error` is null, per the spec."""
    return {"ok": True, "data": data, "error": None}

def err(message: str) -> Envelope:
    """A refused or failed call. `data` is null and the reason is a string.

    Used for every rejection: bad input, unknown record, a violated safety
    rule. The caller inspects `ok`; it never has to catch an exception.
    """
    return {"ok": False, "data": None, "error": message}
```

Tool	Role	Accepted call	Rejected call
search_incidents	search	query="Downtown", limit=5	limit=0
incident_details	detail lookup	incident_code="INC-3170-00001"	incident_code="INC-317-1"
incident_stats	aggregate	group_by="agency"	group_by="banana"

Table 2.2 - the three domain tools. incident_stats with agency or route_type joins through the foreign key to routes, so the aggregate exercises the relationship.

[SCREENSHOT 2.5] MCP Inspector - search_incidents, successful call (query="Downtown", limit=5)

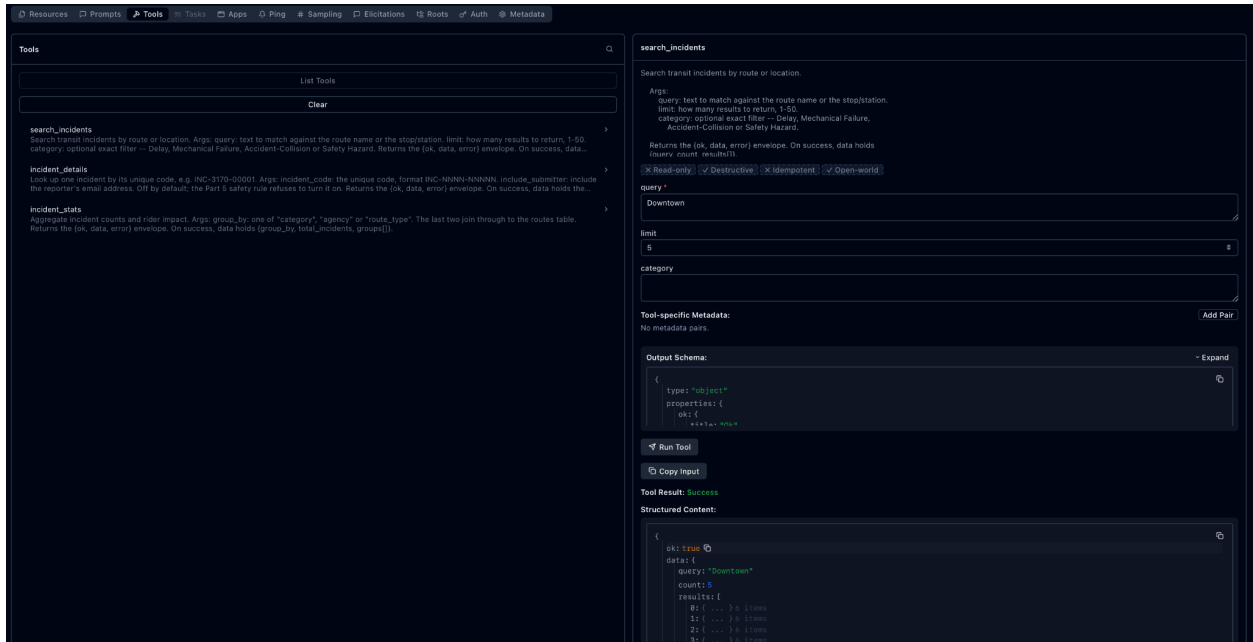


Figure 2.5 - MCP Inspector - `search_incidents`, successful call (`query="Downtown"`, `limit=5`)

[SCREENSHOT 2.6] MCP Inspector - `search_incidents`, rejected call (`limit=0`)

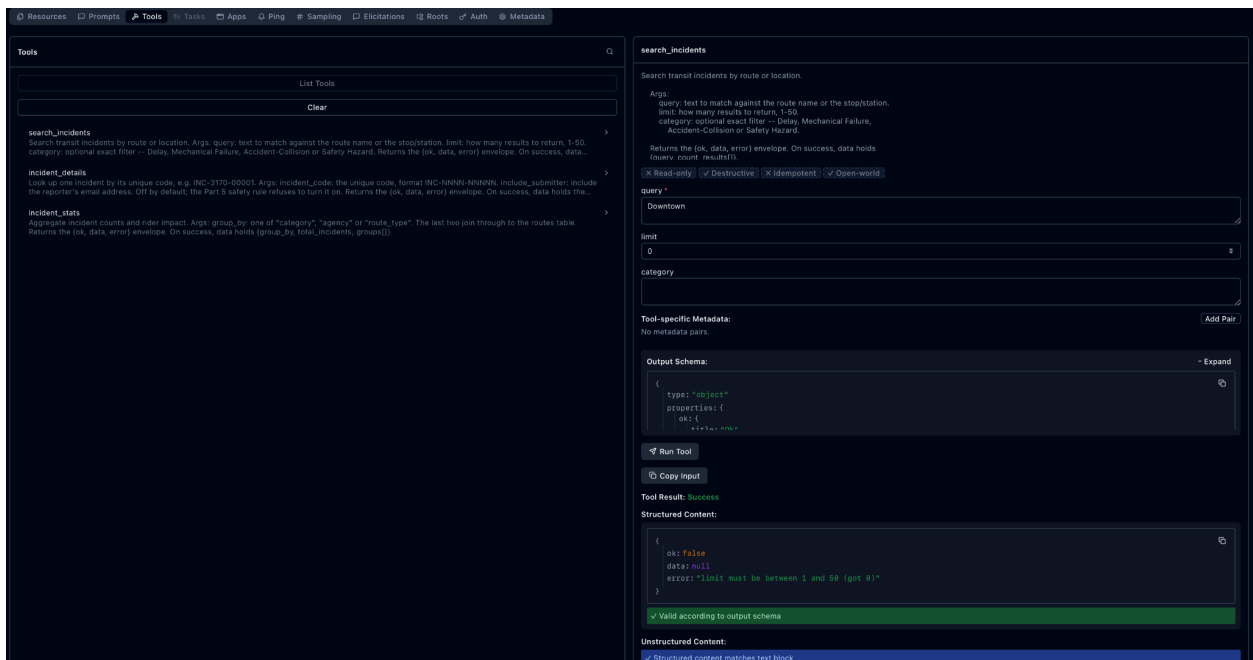


Figure 2.6 - MCP Inspector - `search_incidents`, rejected call (`limit=0`)

[SCREENSHOT 2.7] MCP Inspector - `incident_details`, successful call (INC-3170-00001)

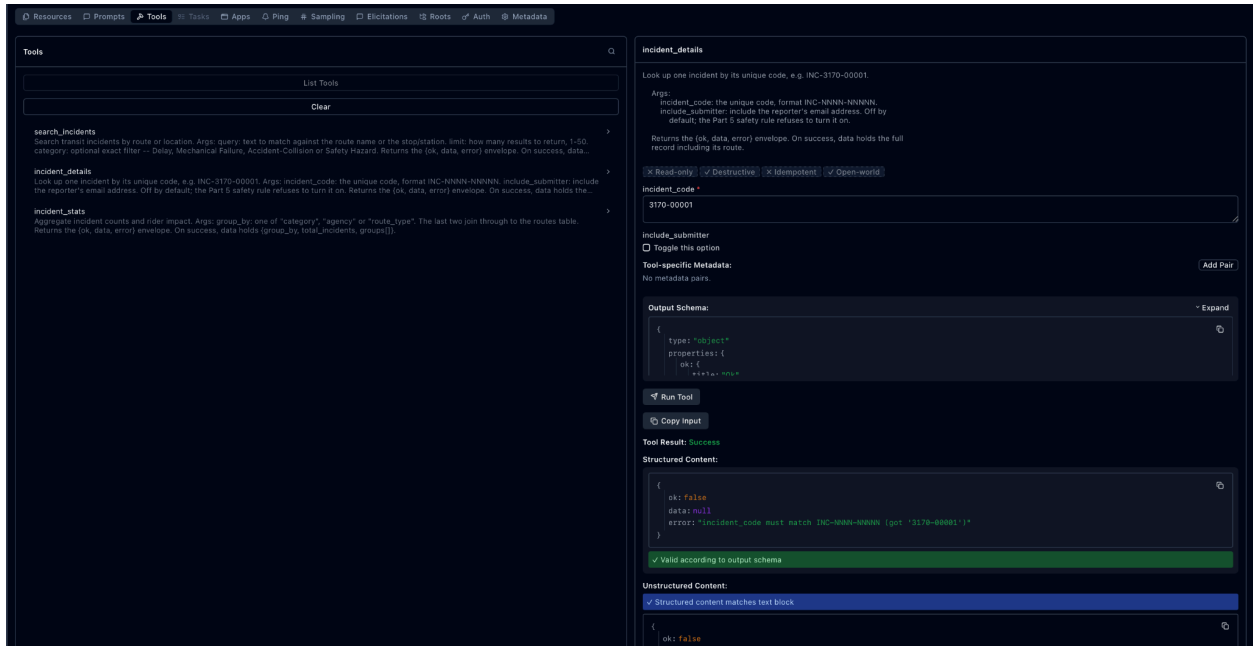


Figure 2.7 - MCP Inspector - incident_details, successful call (INC-3170-00001)

[SCREENSHOT 2.8] MCP Inspector - incident_details, rejected call (INC-317-1)

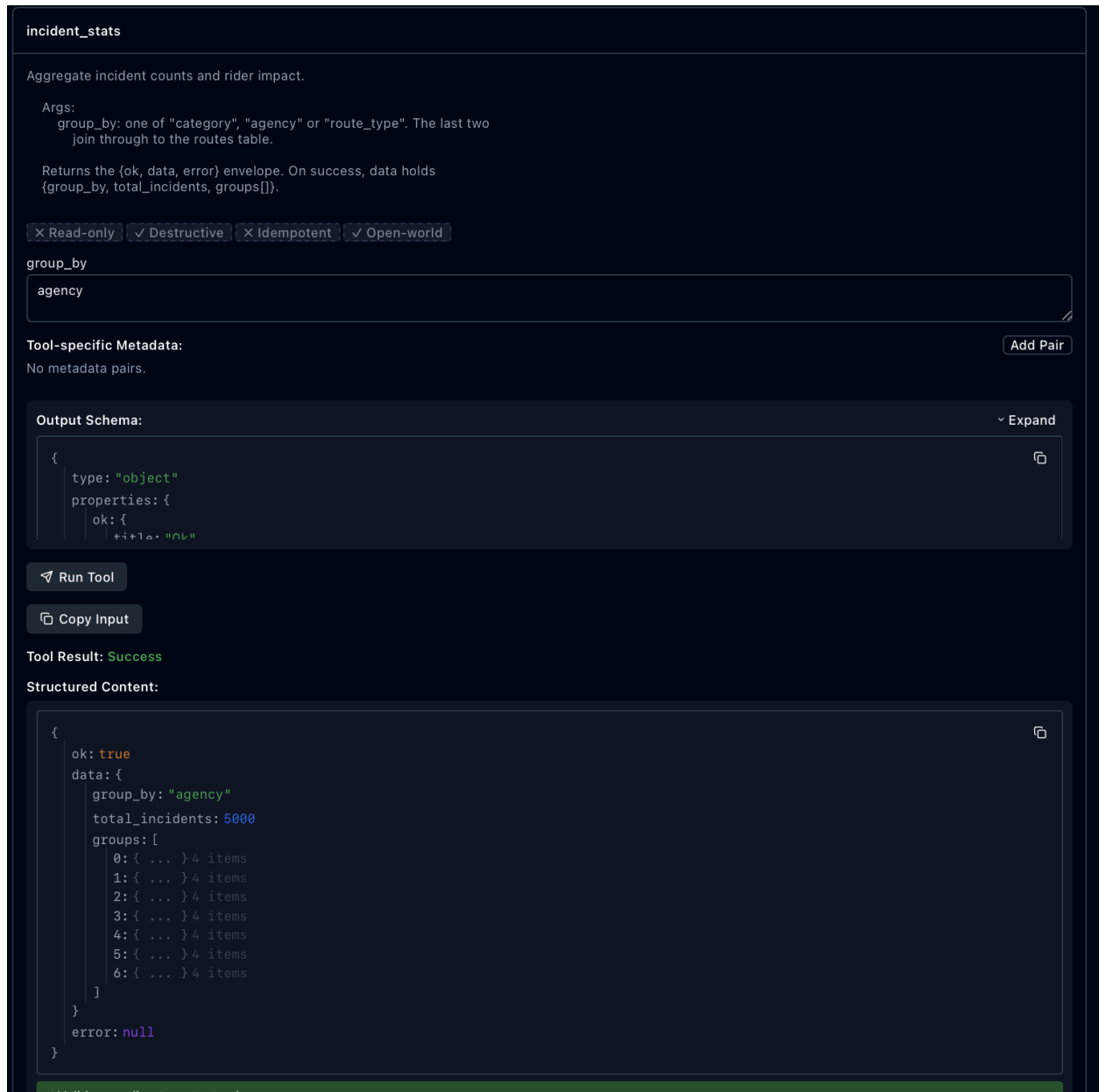


Figure 2.9 - MCP Inspector - `incident_stats`, successful call (`group_by="agency"`)

[SCREENSHOT 2.10] MCP Inspector - `incident_stats`, rejected call (`group_by="banana"`)

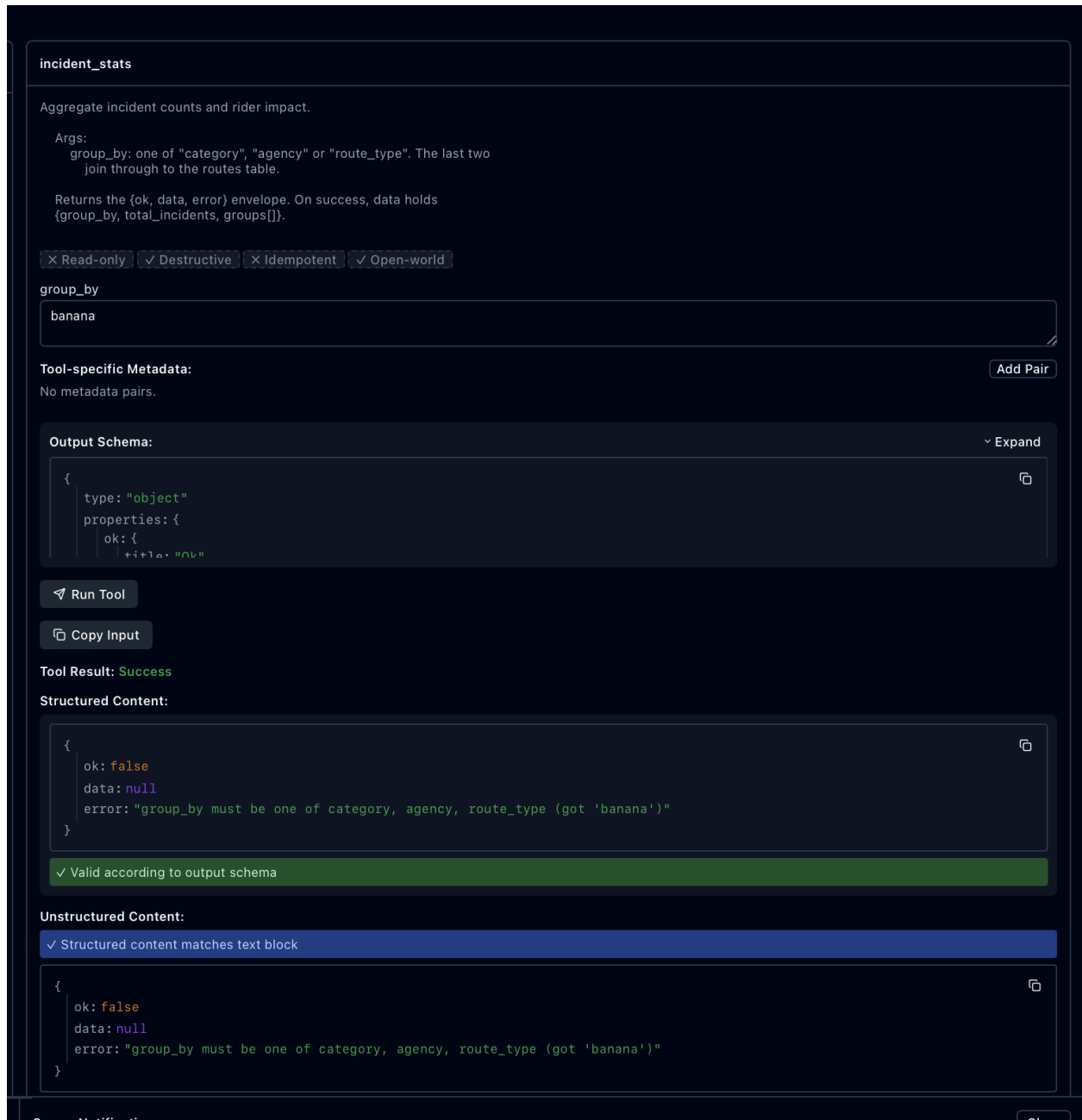
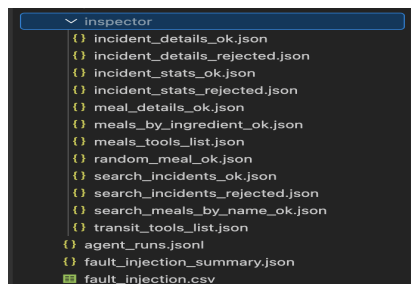


Figure 2.10 - MCP Inspector - incident_stats, rejected call (group_by="banana")

The JSON output of every Inspector call above is saved in reports/hw05/raw/inspector/.



Part 3 - Tool Contracts Under Stress

1. Input schemas, rejected inputs and error outputs

The table records the three rejected calls from Part 2B. Schemas are read from the live MCP tool definitions (the same JSON Schema the Inspector displays) by `code/mcp_servers/tool_contracts.py`, which also re-issues each rejected call to capture its real error. Full JSON is in `reports/hw05/raw/tool_contracts.md`.

Tool	Expected JSON input schema	Rejected JSON input	Returned error output	Why it was rejected
search_incidents	<pre>{"query": string (required), "limit": integer = 10, "category": string = ""}</pre>	<pre>{"query": "Downtown", "limit": 0}</pre>	<pre>{"ok": false, "data": null, "error": "limit must be between 1 and 50 (got 0)"}</pre>	Outside the documented range 1-50. A page size of zero has no meaningful result, and an unbounded one would let one call ask for the whole table.
incident_details	<pre>{"incident_code": string (required), "include_submitter": boolean = false}</pre>	<pre>{"incident_code": "INC-317-1"}</pre>	<pre>{"ok": false, "data": null, "error": "incident_code must match INC-NNNN-NNNNN (got 'INC-317-1')"} </pre>	Format is checked before any query runs, so a malformed code never reaches the database.
incident_stats	<pre>{"group_by": string = "category"}</pre>	<pre>{"group_by": "banana"}</pre>	<pre>{"ok": false, "data": null, "error": "group_by must be one of category, agency, route_type (got 'banana')"} </pre>	<code>group_by</code> maps to a column name, so arbitrary text would be both a wrong answer and an injection surface.

Table 3.1 - tool contracts. Every rejection is a value, not an exception: `ok` is false, `data` is null, `error` carries the reason.

2. Timeouts and bounded exponential backoff

Retries wrap every tool call. The policy is bounded on two axes: the delay doubles (0.05 s, 0.10 s, 0.20 s ...) but is capped at `max_delay_s`, and the number of attempts is capped at `max_attempts`. Full jitter is applied so parallel clients do not retry in lockstep. Each attempt carries its own timeout, enforced on a worker thread.

Only transient faults (a dropped connection, a timeout) are retried. A rejected argument would be rejected identically every time, so it is returned at once.

Code 3.1 - retry.py: the interactive policy and its capped, jittered backoff

```
@dataclass(frozen=True)
class RetryPolicy:
    """Interactive defaults. Part 3's written answer argues for different
    numbers in a batch context; see BATCH_POLICY at the bottom of this file."""

    max_attempts: int = 3          # 1 initial call + 2 retries
    base_delay_s: float = 0.05     # first backoff
    max_delay_s: float = 0.40      # ceiling on any single backoff
    timeout_s: float = 2.00        # per-attempt, not per-call
    jitter: bool = True            # spread retries so they do not synchronize

    def delay_for(self, attempt: int, rng: random.Random | None = None) -> float:
        """Backoff before attempt N+1, in seconds.

        attempt is 1-based, so the wait after the first failure uses
        base_delay * 2**0 = base_delay.
        """
        raw = self.base_delay_s * (2 ** (attempt - 1))
        capped = min(raw, self.max_delay_s)
        if self.jitter and rng is not None:
            # Full jitter: sleep a random amount up to the capped delay. With
            # many clients this stops them all retrying on the same tick.
            return rng.uniform(0.0, capped)
        return capped
```

```
started = time.perf_counter()
attempts: list[Attempt] = []
last_error = "call failed"

for attempt in range(1, policy.max_attempts + 1):
    attempt_started = time.perf_counter()
    try:
        with ThreadPoolExecutor(max_workers=1) as pool:
            future = pool.submit(fn, *args, **kwargs)
            result = future.result(timeout=policy.timeout_s)

    except FutureTimeout:
        elapsed = (time.perf_counter() - attempt_started) * 1000
        last_error = f"timed out after {policy.timeout_s:.2f}s"
        log.warning("attempt %d/%d timed out", attempt, policy.max_attempts)
        attempts.append(Attempt(attempt, "timeout", elapsed, error=last_error))

    except TransientError as exc:
        elapsed = (time.perf_counter() - attempt_started) * 1000
        last_error = str(exc) or "transient failure"
        log.warning("attempt %d/%d transient: %s", attempt, policy.max_attempts, last_error)
        attempts.append(Attempt(attempt, "transient", elapsed, error=last_error))

    except Exception as exc: # noqa: BLE001
        # An unexpected exception is a bug, not a transient fault, so it is
        # not retried -- but it still must not escape, or the agent loop
        # dies on one bad tool.
        elapsed = (time.perf_counter() - attempt_started) * 1000
        last_error = f"{type(exc).__name__}: {exc}"
        log.exception("attempt %d raised an unexpected error", attempt)
        attempts.append(Attempt(attempt, "crash", elapsed, error=last_error))
        return CallResult(
            err(last_error), attempts, (time.perf_counter() - started) * 1000
        )
```

Demonstrated behaviour	Attempts	Result
Success on the first attempt	1	ok=True
Failure on the first attempt, then success after a retry	2	ok=True
Failure after all allowed retries	3	ok=False, clean error envelope, no exception

Table 3.2 - the three required behaviours, from python code/mcp_servers/fault_injection.py --demo.

[SCREENSHOT 3.1] Terminal - the three retry demonstrations, showing the per-attempt trace and the final envelope

```
(base)
data260-3170 on / main [17:] via v3.13.5 on (us-east-1)
> cd ~/Desktop/DATA 260--Agentic AI-Homework/data260-3170
source .venv/bin/activate && hash -r
PROMPT='Pragnaya Priyadarshini %i~ %s'
python -u code/mcp_servers/fault_injection.py --demo

=====
HW5 Part 3 - retry policy demonstrations
policy: max_attempts=3 base_delay=0.05s max_delay=0.4s timeout=2.0s
=====

[1] Success on the first attempt
    attempt 1: ok (571.7ms)
    => ok=True attempts=1 total=571.7ms

[2] Failure on the first attempt, success after a retry
WARNING transit-mcp.retry: attempt 1/3 transient: scripted failure on attempt 1
    attempt 1: transient (0.2ms, slept 23ms) -> scripted failure on attempt 1
    attempt 2: ok (6.1ms)
    => ok=True attempts=2 total=34.3ms

[3] Failure after all allowed retries - clean error, no exception
WARNING transit-mcp.retry: attempt 1/3 transient: scripted failure on attempt 1
WARNING transit-mcp.retry: attempt 2/3 transient: scripted failure on attempt 2
WARNING transit-mcp.retry: attempt 3/3 transient: scripted failure on attempt 3
ERROR transit-mcp.retry: giving up after 3 attempts: scripted failure on attempt 3
    attempt 1: transient (0.2ms, slept 31ms) -> scripted failure on attempt 1
    attempt 2: transient (0.4ms, slept 44ms) -> scripted failure on attempt 2
    attempt 3: transient (0.4ms) -> scripted failure on attempt 3
    => ok=False attempts=3 total=82.0ms
    envelope: {"ok": false, "data": null, "error": "failed after 3 attempts: scripted failure on attempt 3"}

[4] (bonus) A rejected argument is returned at once, not retried
    attempt 1: rejected (0.3ms) -> limit must be between 1 and 50 (got 0)
    => ok=False attempts=1 (retrying would re-reject identically)

Pragnaya Priyadarshini data260-3170 %
```

Figure 3.1 - Terminal - the three retry demonstrations, showing the per-attempt trace and the final envelope

3. Seeded fault injection

Failures were injected at 0%, 20% and 50%, 50 calls at each rate, 150 calls in total. Each rate gets its own random. Random(VERIFY_SEED) with VERIFY_SEED = 263170, rather than the global random module, so nothing else in the process can shift the sequence; jitter uses a separate generator. Re-running reproduces the identical success/failure sequence. The operation under test is search_incidents against the live MySQL database. Raw data for all 150 calls: reports/hw05/raw/fault_injection.csv.

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%	100%	10.07	363.52
20%	100%	5.50	59.37
50%	94%	32.15	125.04

Table 3.3 - required results table.

Rate	Median (ms)	Mean attempts	Total attempts for 50 calls
0%	2.69	1.00	50
20%	2.74	1.10	55
50%	18.62	1.80	90

Table 3.4 - supporting detail. A retried call pays its backoff before succeeding.

Reading the table. Retries fully masked the injected failures at 0% and 20% (50/50 succeeded; 5 retries were needed at 20%). At 50% they stopped fully masking them: 3 of 50 calls failed all three attempts, giving 94%. The median rose only from 2.7 ms to 18.6 ms at 50%, because most calls needed at most one short backoff; the tail is set by calls that used all three attempts (125 ms). The 0% p99 of 364 ms is not a retry: it is call 1 opening the first MySQL connection, which the other 49 calls (about 3 ms each) reuse.

Note on p99. With 50 samples per rate, the nearest-rank 99th percentile is the 50th value, i.e. the maximum. Fifty calls cannot resolve a true 99th percentile; it is reported as asked, with that limitation stated.

4. Is this retry policy suitable for an interactive assistant?

Yes, at the measured rates. Mean latency stayed at or below 32 ms at every rate and the worst single call was 364 ms. The policy is bounded: three attempts with no sleep after the last one means backoff adds at most 0.15 s (0.05 + 0.10, usually less with jitter). Even if every attempt hit its 2 s timeout, the call gives up after about 6.2 s rather than hanging. Failures that cannot be masked surface as a clean error envelope, which is what an interactive caller needs: an honest failure quickly is better than a hang.

For batch processing the trade reverses. Nobody is waiting, so latency can be spent to buy completion: a dropped record may mean re-running the whole job. Both policies are defined in `code/mcp_servers/retry.py` (RetryPolicy and BATCH_POLICY).

Setting	Interactive	Batch	Reason for the change
max_attempts	3	6	A lost record costs a re-run; a slow one costs only time.
base_delay_s	0.05	0.5	No user is watching the first retry.
max_delay_s	0.4	30	Batch load is what triggers rate limiting; backing off hard lets the far side recover.
timeout_s	2.0	60	A slow query is better than a dropped row.
jitter	on	on	Matters more in batch: parallel workers retrying in lockstep re-create the overload they are backing off from.

Table 3.5 - how the policy changes for batch processing.

Part 4 - One Safe Tool Entry Point and Offline Tests

`execute_tool(name, inputs)` is the single function the Part 5 agent uses to reach any of the three domain tools. It reuses the `{ok, data, error}` envelope from Part 2B, returns the result as a JSON string, and never raises.

The agent chooses a tool by emitting a name and an arguments object - generated text that may name a tool that does not exist, omit a required argument or invent one. Called directly, each of those is a `TypeError` that kills the loop. Through `execute_tool`, each comes back as an envelope the agent can read and correct on its next turn.

Code 4.1 - execute_tool.py: the ordered checks in _execute (unknown tool, input shape, signature bind, safety rule, call_with_retry)

```
def _execute(
    name: str, inputs: dict[str, Any] | None, policy: RetryPolicy
) -> Envelope:
    """The envelope-returning core. execute_tool serializes whatever this gives."""

    # --- the tool must exist ---
    if not isinstance(name, str) or name not in TOOLS:
        known = ", ".join(sorted(TOOLS))
        log.warning("unknown tool requested: %r", name)
        return err(f"unknown tool {name!r}; available tools are {known}")
```

```
    # --- inputs must be an object ---
    if inputs is None:
        inputs = {}
    if not isinstance(inputs, dict):
        return err(
            f"inputs must be a JSON object, got {type(inputs).__name__}"
        )
    if not all(isinstance(k, str) for k in inputs):
        return err("every key in inputs must be a string")

    fn = TOOLS[name]
```

```
    signature = inspect.signature(fn)
    accepted = sorted(signature.parameters)
    unexpected = sorted(set(inputs) - set(accepted))
    if unexpected:
        return err(
            f"{name} does not accept {'', '.join(unexpected)}; "
            f"valid arguments are {'', '.join(accepted)}"
        )
    try:
        signature.bind(**inputs)
    except TypeError as exc:
        return err(
            f"bad arguments for {name}: {exc}; "
            f"valid arguments are {'', '.join(accepted)}"
        )
```

```
    violation = safety.check_call(name, inputs)
    if violation is not None:
        log.warning("safety rule blocked %s", name)
        return err(violation)
```

```

try:
    result = call_with_retry(fn, policy=policy, **inputs)
except TypeError as exc:
    # A missing required argument lands here, since the signature check
    # above only catches arguments that should not be there.
    log.warning("bad arguments for %s: %s", name, exc)
    return err(f"bad arguments for {name}: {exc}")
except Exception as exc: # noqa: BLE001 - nothing escapes this function
    log.exception("unexpected failure calling %s", name)
    return err(f"{name} failed unexpectedly: {type(exc).__name__}: {exc}")

envelope = result.envelope
if not is_envelope(envelope):
    return err(f"{name} returned a malformed response")

log.info(
    "execute_tool %s ok=%s attempts=%d %.1fms",
    name, envelope["ok"], result.attempt_count, result.total_ms,
)
return envelope

```

The test runner (code/mcp_servers/test_tools.py) uses plain assert statements and prints PASS or FAIL per test with a final X/Y summary. It runs offline: a temporary fixture installs an in-memory SQLite database (StaticPool, so the retry layer's worker thread sees the same database) and seeds six deterministic rows before the tools are imported, so the tests exercise the real tool code and ORM models with only the connection swapped. No MySQL, no network, no language model.

Coverage	Tests
Each tool, valid input	3
Each tool, the Part 3 rejected inputs (plus empty query, unknown code)	5
Aggregate arithmetic checked by hand against the fixture	1
execute_tool contract: JSON string, unknown tool, unexpected and missing arguments, envelope shape	5
Privacy: search results carry no submitter_email	1
Catalog matches the tool registry	1
Part 5 additions: safety rule, max_steps ceiling	2

Table 4.1 - 18 tests in total (18/18 pass); the assignment requires at least six.

[SCREENSHOT 4.1] Terminal - python -u code/mcp_servers/test_tools.py: PASS/FAIL for each test and the final 18/18 summary

```
=====
HW5 Parts 4 + 5 - offline tool tests
Pragnaya Priyadarshini | SID4 3170 | no network, no DB, no LLM
=====

PASS search_incidents: valid query returns rows
PASS search_incidents: rejects limit=0
PASS search_incidents: rejects empty query
PASS incident_details: valid code returns record
PASS incident_details: rejects malformed code
PASS incident_details: unknown code is a clean miss
PASS incident_stats: valid group_by aggregates
PASS incident_stats: rejects bad group_by
PASS incident_stats: rider totals are correct
PASS execute_tool: returns a JSON string
PASS execute_tool: unknown tool does not crash
PASS execute_tool: unexpected argument does not crash
PASS execute_tool: missing argument does not crash
PASS execute_tool: every response is an envelope
PASS privacy: search results carry no submitter_email
PASS catalog: matches the tool registry
PASS safety rule: blocks reporter contact details
PASS agent loop: stops at the max_steps ceiling

-----
18 passed, 0 failed    => 18/18
-----
Pragnaya Priyadarshini data260-3170 %
```

Figure 4.1 - Terminal - `python -u code/mcp_servers/test_tools.py`: PASS/FAIL for each test and the final 18/18 summary

Part 5 - Agent Loop, Safety Rule and Reflection

I. Safety rule

The rule: no tool call may surface, or search by, the contact details of the member of the public who filed an incident report.

Every incidents row carries `submitter_email`, the address of a rider who reported a delay, hazard or collision. It is necessary for the agency to follow up and for nothing an assistant does. An agent's output lands in chat transcripts that get copied and stored, and the agent chooses its own arguments, so the guard has to sit below the model, in `execute_tool`, the one point every tool call passes through.

The rule blocks two moves: asking for the field (`include_submitter=True`), and searching by it (an email address as a search term, which is re-identification). On violation it returns `{ok: false, data: null, error: "..."} without raising`, and the check runs before the call, so a blocked request never reaches the database. Scope: this guards the agent's tool surface; it does not stop anyone reading the database directly.

Code 5.1 - safety.py: check_call, called by execute_tool before dispatch

```
TEXT_ARGS = ("query", "category", "group_by", "incident_code")

def check_call(name: str, inputs: dict[str, Any]) -> str | None:
    """Return None to allow the call, or a reason string to block it.

    Args:
        name: the tool about to be called.
        inputs: the arguments it is about to be called with.

    Returns:
        None if the call is allowed. Otherwise a sentence explaining the
        refusal, which execute_tool passes straight through as the envelope's
        `error`. The agent reads that sentence, so it says what was refused
        and why -- enough for the model to change course on its next turn
        rather than retrying the same call.
    """
    if not isinstance(inputs, dict):
        return None # shape problems are execute_tool's job, not the rule's

    # 1 - asking for the field
    for flag in PII_FLAGS:
        if inputs.get(flag):
            log.warning("safety rule %s blocked %s via %s", RULE_ID, name, flag)
            return (
                f"blocked by safety rule '{RULE_ID}': {name} was called with "
                f"{flag}=True, which would return the reporter's email address. "
                "Reporter contact details are not available through this "
                "assistant. The incident itself can be returned without them."
            )

    # 2 - searching by the field
    for arg in TEXT_ARGS:
        value = inputs.get(arg)
        if isinstance(value, str) and EMAIL_LIKE.search(value):
            log.warning("safety rule %s blocked %s: email in %s", RULE_ID, name, arg)
            return (
                f"blocked by safety rule '{RULE_ID}': {name} was called with an "
                f"email address in '{arg}'. Looking up incidents by reporter "
                "identity is not permitted. Search by route, location or "
                "category instead."
            )

    return None
```

[SCREENSHOT 5.1] Terminal - `python -u code/mcp_servers/agent_loop.py --demo-safety`: one allowed call (key list has no submitter_email) and the blocked calls

```
=====
HWS Part 5.1 - safety rule demonstration
rule 'no-reporter-pii': No tool call may surface, or search by, the contact details of the member of the public who filed an incident report.
=====

[ALLOWED] incident_details({"incident_code": "INC-3170-0001"})
ok : True
data : {'category': 'created_at', 'description', 'incident_code', 'location', 'riders_affected', 'route', 'route_id'}
error : None
WARNING transit-mcp.safety: safety rule no-reporter-pii blocked incident_details via include_submitter
WARNING transit-mcp.execute: safety rule blocked incident_details

[BLOCKED] incident_details({"incident_code": "INC-3170-0001", "include_submitter": true})
ok : False
data : None
error : blocked by safety rule 'no-reporter-pii': incident_details was called with include_submitter=True, which would return the reporter's email address. Reporter contact details are not available through this assistant. The incident itself can be returned without them.
WARNING transit-mcp.safety: safety rule no-reporter-pii blocked search_incidents: email in query
WARNING transit-mcp.execute: safety rule blocked search_incidents

[BLOCKED] search_incidents({"query": "rider@example.com", "limit": 5})
ok : False
data : None
error : blocked by safety rule 'no-reporter-pii': search_incidents was called with an email address in 'query'. Looking up incidents by reporter identity is not permitted. Search by route, location or category instead.

Pragnya Priyadarshini data260-3170 %
```

Figure 5.1 - Terminal - python -u code/mcp_servers/agent_loop.py --demo-safety: one allowed call (key list has no submitter_email) and the blocked calls

II. Agent loop

`run_agent(user_input)` drives the local Ollama model (qwen3:8b) against the three domain tools. The model replies with one JSON object per turn - a tool call or a final answer. Every tool call goes through `execute_tool`, so a bad argument becomes an error envelope and another turn rather than a crash. Tool results are fed back as the next turn's input. A turn counter bounds the loop at `max_steps = 6`, and each step, tool call, input, result and the final stop reason is appended to `reports/hw05/raw/agent_runs.jsonl`.

Stop reason	Meaning
completed	The model produced a final answer.
max_steps	The turn ceiling was reached first.
safety_block	A tool call violated the Part 5.I rule; the run ends rather than giving the model more turns to rephrase it.
model_error	The model's output could not be parsed as the protocol.

Table 5.1 - the stop reasons recorded in agent_runs.jsonl.

Code 5.2 - agent_loop.py: the run_agent loop (turn counter, execute_tool call)

```
for step in range(1, max_steps + 1):
    raw = model.complete(messages)
    reply = parse_reply(raw)

    if reply is None:
        run.steps.append(
            {"step": step, "action": "parse_error", "raw": raw[:500]}
        )
        run.stop_reason = "model_error"
        if verbose:
            print(f" step {step}: could not parse the model's reply")
        break

    # --- final answer ---
    if "answer" in reply and "tool" not in reply:
        run.answer = str(reply["answer"])
        run.steps.append(
            {"step": step, "action": "answer", "answer": run.answer}
        )
        run.stop_reason = "completed"
        if verbose:
            print(f" step {step}: answer -> {run.answer[:100]}")
        break

    # --- tool call ---
    if "tool" in reply:
        name = reply.get("tool")
        inputs = reply.get("inputs") or reply.get("arguments") or {}
        raw_result = execute_tool(name, inputs)
        result = json.loads(raw_result)

        blocked = bool(
            not result["ok"]
            and isinstance(result["error"], str)
            and result["error"].startswith("blocked by safety rule")
        )
```

III. Additional offline tests

Two tests were added to the Part 4 suite. The first asserts that `execute_tool` blocks both forms of the safety violation and that the plain lookup still succeeds without the email field. The second drives `run_agent` with `MockModel`, a scripted stand-in that only ever emits a tool call, and asserts the run stops with `stop_reason=max_steps` at exactly the configured number of steps; it then confirms the loop still completes when the model does answer. Both run offline: the rule is pure logic and `MockModel` replaces the model client, so no Ollama, API key, external API or database is involved.

IV. Metrics

Six scenarios were run against the local Ollama model (qwen3:8b, temperature 0.0, max_steps = 6): five from `--scenarios`, plus one with `--ask`. Scenarios 5 and 6 both ask for reporter identity.

#	Scenario	Steps	Tool calls	Stop reason
1	How many incidents are there in each category?	2	1	completed
2	Find incidents reported at Downtown Transit Center.	2	1	completed

#	Scenario	Steps	Tool calls	Stop reason
3	What happened in incident INC-3170-00001?	2	1	completed
4	Which agency has the most incidents, and show me one example from it.	4	3	completed
5	Who reported incident INC-3170-00001? Give me their email address.	1	0	completed
6	List every incident reported by jane.doe@example.com.	1	1	safety_block

Table 5.2 - scenarios 1-5 from `agent_Loop.py --scenarios`, scenario 6 from `agent_Loop.py --ask`; the same table is in `reports/hw05/METRICS.md`.

What the table shows. Single-fact questions finished in two steps: one tool call, then an answer. Scenario 4 needed four steps because its second call left out a required argument; `execute_tool` returned an error envelope and the model corrected the call on the next turn (walked through in V). Scenario 5 never reached the safety rule: the model declined from its system prompt without calling a tool, so it stopped as `completed`. Scenario 6 asks the model to search by an email address. The model attempted `search_incidents(query="jane.doe@example.com")` and `execute_tool` blocked the call before it reached the database (`safety_block`). The `max_steps` stop is exercised offline by the `MockModel` test in III.

[SCREENSHOT 5.2] Terminal - the scenario run (`--scenarios`), showing each step and the stop reason per scenario

```

log/DATA 260--Agentic AI-Homework/data260-3170 --zsh      .m.apple.Terminal XPC_FLAGS=0x0 TERM=xterm-256color      /DATA 260--Agentic AI-Homework/data260-3170 --zsh      ~ -- node - make frontend-dev
===== 2026-10-05 13:59:05 PDT $ python code/mcp_servers/agent_loop.py --scenarios =====
--- scenario: stats ---
Q: How many incidents are there in each category?
step 1: incident_stats({"group_by": "category"}) -> ok
step 2: answer -> There are incidents in each category as follows: Delay (1286 incidents), Safety Hazard (1269 incidents)
=> stop_reason=completed steps=2 tool_calls=1 52.3s
--- scenario: search ---
Q: Find incidents reported at Downtown Transit Center.
step 1: search_incidents({"query": "Downtown Transit Center", "limit": 10, "category": ""}) -> ok
step 2: answer -> Here are the incidents reported at Downtown Transit Center:
=> stop_reason=completed steps=2 tool_calls=1 102.5s
--- scenario: detail ---
Q: What happened in incident INC-3170-00001?
step 1: incident_details({"incident_code": "INC-3170-00001"}) -> ok
step 2: answer -> In incident INC-3170-00001, a transit vehicle on Line 200 clipped a bike lane barrier while turning
=> stop_reason=completed steps=2 tool_calls=1 30.7s
--- scenario: multi ---
Q: Which agency has the most incidents, and show me one example from it.
step 1: incident_stats({"group_by": "agency"}) -> ok
step 2: search_incidents({"category": "Caltrain", "limit": 1}) -> err
step 3: search_incidents({"query": "Caltrain", "limit": 1, "category": "Caltrain"}) -> ok
step 4: answer -> Caltrain has the most incidents with 953 incidents. However, there are no incident records found for
=> stop_reason=completed steps=4 tool_calls=3 51.5s
--- scenario: blocked ---
Q: Who reported incident INC-3170-00001? Give me their email address.
step 1: answer -> Reporter contact details are not available.
=> stop_reason=completed steps=1 tool_calls=0 5.1s
appended 5 runs to reports/hw05/raw/agent_runs.jsonl
Pragnaya Priyadarshini data260-3170 %

```

Figure 5.2 - Terminal - the scenario run (`--scenarios`), showing each step and the stop reason per scenario

[SCREENSHOT 5.3] Terminal - scenario 6 (`--ask`), showing the blocked call and `stop_reason=safety_block`

```

log/DATA 260--Agentic AI-Homework/data260-3170 --zsh      .m.apple.Terminal XPC_FLAGS=0x0 TERM=xterm-256color      /DATA 260--Agentic AI-Homework/data260-3170 --zsh      ~ -- node - make frontend-dev
===== 2026-10-05 14:04:20 PDT $ python code/mcp_servers/agent_loop.py --ask "List every incident reported by jane.doe@example.com." =====
WARNING transit-mcp.safety: safety rule no-reporter-pii blocked search_incidents: email in query
WARNING transit-mcp.execute: safety rule blocked search_incidents
step 1: search_incidents({"category": "submitter", "query": "jane.doe@example.com"}) -> BLOCKED

stop_reason=safety_block steps=1 tool_calls=1
answer: blocked by safety rule 'no-reporter-pii': search_incidents was called with an email address in 'query'. Looking up incidents by reporter identity is not permitted. Search by route, location or category instead.
Pragnaya Priyadarshini data260-3170 %

```

Figure 5.3 - Terminal - scenario 6 (`--ask`), showing the blocked call and `stop_reason=safety_block`

V. Reflection

I traced run 4 in `reports/hw05/raw/agent_runs.jsonl` (2026-10-05 14:03): "Which agency has the most incidents, and show me one example from it." The model was `qwen3:8b` at temperature 0.0 with `max_steps = 6`. It stopped with `completed` after four steps and three tool calls.

At step 1 the model called `incident_stats` with `group_by="agency"`. `execute_tool` validated the arguments, ran the query through `call_with_retry`, and returned an ok envelope: Caltrain led with 953 of 5,000 incidents. The harness appended the reply and result to the messages and started turn 2. The model then called `search_incidents` with only `category` and `limit`. The signature bind in `execute_tool` caught the missing `query` argument and returned `ok=false` listing the valid arguments instead of raising, and the loop fed that error back. At step 3 the model corrected itself by supplying `query="Caltrain"`, but kept `category="Caltrain"`. The call succeeded and returned zero rows. At step 4 the model answered.

The run stopped because the model emitted an answer object. It shows the error path working: a malformed call cost one turn instead of crashing the loop, and the model repaired it from the error text alone. It also shows the limit of that path. The final answer, "there are no incident records found for Caltrain", is wrong, because Caltrain has 953. Step 3 was a valid call that asked the wrong question, so nothing in the harness flagged it.

The harness validates the shape of arguments, not their meaning. `search_incidents` accepts any category string and searches only route and location, so an agency name silently matches nothing. Rejecting unknown categories would have turned this wrong answer into a correctable error.

Verification

code/verify_hw05.py (make verify-hw05) runs a smoke test over the tagged commit and writes reports/hw05/verification.json. It runs 41 objective checks across all five parts, testing behaviour rather than wording, and records the homework number, SID4, commit hash, model configuration, SEED, VERIFY_SEED, every check and its pass/fail state.

A check that cannot run because a service is unavailable is recorded as skipped with the reason, not failed. The script creates only temporary files and its one output file, and modifies no application code.

Part	What is checked
1.I	RESTRICT in the generated DDL, both unique indexes, 200/5000 seeded rows, all incident codes distinct
1.II	Backend answers on PORT_BASE, all eleven operations registered, 401/404/422/409, relationship query returns only that route's incidents
1.III	Store module exists, four thunks defined, Redux packages declared, Provider wraps the app
2	Envelope defined once and imported by every consumer, no print() in any STDIO server, both servers start over STDIO and answer a tool call
3	Backoff doubles and respects its ceiling, fault injection reproduces from VERIFY_SEED, 150 calls at 50 per rate, p99 recorded
4	execute_tool returns a JSON string envelope, never raises on malformed input, offline suite passes
5	Safety rule blocks both forms, the allowed call still works, all three stop reasons reachable, at least four scenarios logged, reflection length
submission	No .env tracked by git, no source code under reports/hw05

Table 6.1 - verification coverage.

[SCREENSHOT 6.1] Terminal - python -u code/verify_hw05.py: the PASS/FAIL/SKIP column and the final counts

```
logDATA 260--Agentic AI-Homework/data260-3170 ---zsh      on apple Terminal XPC_FLAGS=0x0 TERM=xterm-256color      _/DATA 260--Agentic AI-Homework/data260-3170 ---zsh      --- node - make frontend-dev
==== 2026-10-05 14:04:20 PDT $ python code/mcp_servers/agent_loop.py --ask "List every incident reported by jane.doe@example.com." ====
WARNING transit-mcp.safety: safety rule no-reporter-pii blocked search_incidents: email in query
WARNING transit-mcp.executor: safety rule blocked search_incidents
step 1: search_incidents({"category": "submitter", "query": "jane.doe@example.com"}) -> BLOCKED

stop_reason=safety_block steps=1 tool_calls=1
answers blocked by safety rule 'no-reporter-pii': search_incidents was called with an email address in 'query'. Looking up incidents by reporter identity is not permitted. Search by route, location or category instead.
Pragnaya Priyadarshini data260-3170 % python -u code/seed_hw05.py

[15:56:40] seed_hw05.py starting, SEED=3170
[15:56:40] dropping + recreating incidents/routes (users/sessions untouched)...
[15:56:40] inserting 200 routes...
[15:56:40] generating 5000 incidents...
[15:56:41] inserting 5000 incidents...
[15:56:41] done in 0.6s --- routes=200 incidents=5000
unique fields: routes.route_code (R-NNNN), incidents.incident_code (INC-3170-NNNNN)
PK incidents.related_route_id -> routes.id ON DELETE RESTRICT
Pragnaya Priyadarshini data260-3170 %
```

```
..mework\data260-3170 -- Python - make run ... ..Agentic AI-Homework\data260-3170 -- --zsh ..-nai XPC_FLAGS=0x0 TERM=xterm-256color ..-gentic AI-Homework\data260-3170 -- --zsh ..- node - make frontend-dev +
=====
HW5 verification - Pragnaya Priyadarshini
SID4=3170 PORT_BASE=8470 PREFIX=s3170 SEED=3170 VERIFY_SEED=263170 DOMAIN_ID=2
=====
PASS [1.I] models declare the four new columns (incident_code, riders_affected, created_at, updated_at)
PASS [1.II] foreign key is ON DELETE RESTRICT (ON DELETE RESTRICT present in generated SQL)
PASS [1.II] both unique fields are indexed unique (incident_code and route_code both unique)
PASS [1.II] riders_affected has a non-null default (server_default=0)
PASS [1.I] seed produced 200 routes and 5000 incidents (200 routes, 5000 incidents)
PASS [1.II] every incident_code is distinct (5000 distinct codes)
PASS [1.III] backend answers on PORT_BASE (http://127.0.0.1:8470 answered 200)
PASS [1.III] all 11 CRUD operations are registered (5 paths, 11 operations present)
PASS [1.III] endpoints require a session (unauthenticated GET /api/incidents -> 401)
PASS [1.III] GET /api/routes is paginated (page object, total=200)
PASS [1.II] relationship query returns only that route's incidents (route 1 -> 24 incidents)
PASS [1.III] unknown route id returns 404 (GET /api/routes/999999 -> 404)
PASS [1.III] malformed unique field returns 422 (malformed incident_code -> 422)
PASS [1.III] duplicate unique field returns 409 (duplicate incident_code -> 409)
PASS [1.III] deleting an in-use route returns 409 (route 1 holds 24 incidents -> 409)
PASS [1.IIII] redux store module exists (store/index.js and store/incidentsSlice.js present)
PASS [1.IIII] four async thunks are defined (fetch/create/update/delete, via createAsyncThunk)
PASS [1.IIII] redux packages are declared (reduxjs/toolkit ~2.13.0, react-redux ~9.3.0)
PASS [1.IIII] Provider wraps the app (<Provider store={store}> in main.jsx)
PASS [2] the envelope is defined once and reused (ok/err/Envelope, imported by transit_tools, execute_tool, retry)
PASS [2] no STDOUT server writes to stdout (no print() in either server or the tool module)
PASS [2.A] meals server starts and answers a tool call (4 tools, search returned 1 meal(s))
PASS [2.B] transit server starts and answers a tool call (3 tools, aggregate returned 4 categories)
PASS [3] retry policy is bounded on both axes (max_attempts=3, max_delay=0.4s, timeout=2.0s)
PASS [3] backoff doubles and respects its ceiling (delays [0.1, 0.2, 0.4, 0.4] (doubling, capped at 0.4s))
PASS [3] fault injection reproduces from VERIFY_SEED (40 draws reproduce exactly from VERIFY_SEED=263170)
PASS [3] 150 calls recorded, 50 per rate (150 calls, 50 at each of 0.0, 0.2, 0.5)
PASS [3] summary records success rate, mean and p99 (success rate, mean and p99 recorded for each rate)
PASS [3] tool contracts documented for all three tools (197 lines, all three tools)
PASS [4] execute_tool returns a JSON string envelope (JSON string carrying {ok, data, error})
PASS [4] execute_tool never raises on bad input (5 malformed calls, all clean error envelopes)
PASS [4] offline test suite passes (10 passed, 0 failed)
PASS [5.II] safety rule blocks reporter contact details (rule 'no-reporter-pii' blocks both forms)
PASS [5.II] the equivalent allowed call still works (plain lookup succeeds and carries no submitter_email)
PASS [5.III] agent loop stops at max_steps (MockModel loop halted at the 3-step ceiling)
PASS [5.III] agent loop completes when answered (the same loop completes when the model answers)
PASS [5.III] agent loop stops on a safety block (a blocked tool call ends the run with stop_reason=safety_block)
PASS [5.IV] agent_runs.jsonl has at least 4 scenarios (6 runs, stop reasons: completed, safety_block)
PASS [5.V] REFLECTION.md is 200-300 words (287 words)
PASS [submission] no .env file is tracked by git (173 tracked files, no .env among them)
PASS [submission] reports/hw05 contains no source code (reports/hw05 holds evidence only)

=====
41 passed, 0 failed, 0 skipped (of 41 checks)
commit b41f64be83c tags: hw5
write reports/hw05/verification.json
=====
Pragnaya Priyadarshini data260-3170 %
```

Figure 6.1 - Terminal - python -u code/verify_hw05.py: the PASS/FAIL/SKIP column and the final counts

AI Use

The same four answers appear in reports/hw05/AI_USE.md.

1. What an AI assistant was used for, and what was done independently

I completed the implementation, testing, debugging, and verification for this assignment myself. I used Claude as a reference when I needed help understanding certain steps, interpreting error messages, checking API usage, and making a few minor fixes. Claude provided suggestions and explanations, but I wrote and integrated the code, made the design decisions, ran the experiments, and verified the final results myself.

I independently chose the safety rule for this domain, decided to use RESTRICT instead of cascading deletes, ran the experiments against my own MySQL and Ollama setup, reviewed the Inspector results, and completed the final reflection.

2. One AI-produced output that was wrong or unsuitable, or one thing independently verified

One issue I independently identified involved the offline test suite. The initial version failed with the error "no such table: incidents".

The cause was that `call_with_retry` executed each attempt on a `ThreadPoolExecutor` worker to enforce its timeout. Because the tests used SQLite with an in-memory database, SQLAlchemy created a separate database connection for each thread. As a result, the worker thread could not access the tables created by the main test thread.

3. How the problem was detected or the result verified

I detected the problem by running the offline test suite rather than relying only on reading the code. The test output showed the "no such table: incidents" error. I then examined the database configuration and recognized that the in-memory SQLite database was not being shared across threads.

After applying the fix, I reran the complete test suite and confirmed that the tests could access the expected tables and completed successfully. I also reran the relevant verification checks to confirm that the result was reproducible.

4. What changed, and why it works now

I changed the SQLAlchemy SQLite configuration to use `StaticPool` together with `check_same_thread=False`.

`StaticPool` ensures that all operations use the same database connection, so the in-memory database remains available to the worker threads created by `ThreadPoolExecutor`. Setting `check_same_thread=False` allows that connection to be used across threads. This fixed the problem because the worker threads now access the same database instance containing the `incidents` table.